第一部分　初识 Python 语言

本部分主要讲解初识 Python 语言的那些事儿，让读者能够快速入门，跨越这个最朦胧、最期待的未知地带，了解 Python 语言的基本概念并建立对编程方法的基本理解。

本部分的学习目标是编写 10 行左右的 Python 程序，共包括 3 章（第 1、2 章和附录 A），分别如下：

第 1 章　程序设计基本方法

第 2 章　Python 基本语法元素

附录 A　极简计算机基础

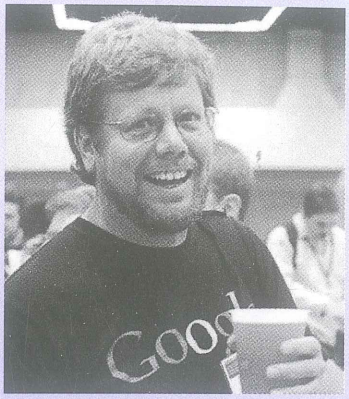
第 1 章主要面向初学编程语言的读者，重点讲解编写程序最基本的 IPO 方法，介绍 Python 语言安装和运行过程，讲解计算思维、计算问题和算法，体验一批 Python 小程序，讲解“流程表达”计算方法，构建最基本且必要的程序设计理念。

第 2 章讲解两个 Python 程序实例，围绕实例介绍 Python 语言的基本语法元素，讲解 Python 库及引用方法，介绍 Python 语言的三种编程模式：过程式、生态式和对象式，建立编写 10 行左右代码的基本信心。

附录 A 补充介绍编写程序所需了解的计算机基础概念，包括：数值和数据、计算机硬件、计算机软件、计算机网络、常用 Python 编辑器，采用极简叙述方式撰写，帮助读者建立对计算机基础概念的简要认识。

除附录 A 作为参考外，建议按照章节顺序逐步学习。对于有一定程序设计基础但不熟悉 Python 语言的读者，建议了解 Python 语言安装环境后重点学习第 2 章内容。

吉多·范罗苏姆：Python 之父



吉多·范罗苏姆（Guido van Rossum）于 1956 年 1 月 31 日出生在荷兰哈尔勒姆，是 Python 程序设计语言的创造者。他在阿姆斯特丹自由大学学习期间，对计算机科学特别是算法和数据结构有了很深入的理解，这为他后续的编程生涯奠定了坚实基础。

1980 年，吉多加入荷兰国家研究院并参与了名为“ABC”的项目，旨在开发一种易于学习和使用的编程语言 ABC。尽管 ABC 语言有其优点，但其局限性也很冲突，这些局限性促使吉多萌生了创造新语言的想法。1989 年冬天，吉多在圣诞假期期间开始设计一种新的编程语言，这就是后来享誉全球的 Python。吉多希望 Python 能够结合 ABC 语言的易用性和 C 语言的强大功能，成为一门既适合初学者，又能够满足专业开发者需求的编程语言。

吉多不仅是一位杰出的程序员，更是一位优秀的领导者。他以开放的态度和谦逊的个性赢得了全球开发者的尊敬。他总是鼓励开发者提出建议和反馈，这种开放的沟通风格使得 Python 社区始终充满活力。吉多的贡献不仅限于 Python 语言本身，他还积极参与了多项开源项目和技术标准的制定。由于其在计算机科学领域的杰出贡献，吉多获得了多项荣誉，于 2006 年被美国计算机协会（ACM）授予著名工程师称号，于 2016 年被评为 IEEE Fellow。这些荣誉充分肯定了他在计算机科学和工程领域所作的突出贡献。

Python 如今已经成为全球最受欢迎的编程语言之一，广泛应用于 Web 开发、数据分析、人工智能、科学计算等众多领域。这一切都离不开吉多的辛勤付出和坚定信念。他不仅创造了一门语言，更构建了一个强大而活跃的社区。在离开核心开发者团队后，吉多仍然活跃于技术社区，继续为 Python 的发展贡献力量。吉多的故事激励着新一代的程序员，激励他们在编程和技术创新的道路上不断追求卓越。他的贡献深刻地影响了整个计算机科学领域，他的故事是一段关于创造、坚持与创新的传奇，永远激励着未来的技术创新者。

第 1 章　程序设计基本方法

在这个国家，每个人都应该学习如何编程，因为它教你如何思考。

Everybody in this country should learn how to program a computer, because it teaches you how to think.

——史蒂夫·乔布斯（Steve Jobs）
苹果公司创始人

学习目标

（1）了解计算机技术的发展历程。
（2）了解 Python 语言的发展及特点。
（3）掌握 Python 开发环境的配置方法。
（4）掌握 IPO 程序编写方法。
（5）理解算法与计算思维的概念。
（6）掌握简单 Python 程序的运行方法。
（7）了解编程语言的两种执行方式。

Hello World 是 1978 年 Brian Kernighan 经典著作《C 程序设计语言》的第一个例子，时至今日，它逐渐演变成具有象征意义的里程碑，几乎成为学习每门编程语言不可替代的首个程序。接下来，读者将看到如何用 Python 语言来编写简洁、优美的 Hello World 程序。

向计算机世界发出你的问候吧！

微视频 1-1：计算机与程序设计



1.1 计算机的概念

要点： 计算机是根据指令操作数据的设备，具备功能性和可编程性两个基本特性。

计算机，不可否认，是人类最伟大的发明之一。

英文“Computer”，最初指专门负责计算的人，类似售票员，这是一个工作岗位。到了 20 世纪中期逐渐演变为计算设备，当代特指电子计算机。

人类为何需要计算？

计算机的故事要从人类始于久远但延续至今的计算需求说起。很显然，人类在敬畏自然、认识自然甚至试图驾驭自然的过程中，为了认识自然现象、分析自然规律，需要进行量化计算。人类社会对有限资源的分配、对人类活动的有效管理，需要进行优化计算。人类探索思维空间的数学、逻辑和哲学问题，需要进行推演计算。到了 21 世纪，人类间便捷和高效的通信需求推动了网络计算的发展，计算需求已经深入到人类社会的方方面面，无处不在。

求解计算问题的方法由计算科学来研究，具体的计算任务由计算设备来完成。广义上讲，计算设备包含但不限于计算机。

计算机的定义有很多种，如下定义更符合计算机的本质：

计算机是根据指令操作数据的设备（a computer is a machine that manipulates data according to a list of instructions）。

从定义可以看出，计算机有两个基本特性：功能性和可编程性。功能性指对数据的操作，表现为数据输入、数据计算和结果输出等。可编程性指对计算机的控制，根据一系列指令自动地、可预测地、准确地完成编程者的意图。

理解和认识计算机应结合上述两个特性。一般意义上，只要设备具备了计算的功能性和操作的可编程性，就可以看作是计算机。判断一个计算设备是否属于计算机并不依靠其制造材质，计算机不一定是电子的。例如，前沿领域的光计算机、量子计算机、超导计算机、生物计算机等新形态计算设备并不建立在电子学基础上，但它们都体现了计算功能和可编程特性，也属于计算机类别。鉴于本书内容的专属性，后续内容的“计算机”均指电子计算机。

计算机技术以性能为核心驱动力，功能性和可编程性交替演进，构成发展路径。

计算机硬件所依赖的集成电路规模按照“摩尔定律”以指数方式增长，计算机运行速度也接近几何级数快速增加。一方面，性能提高使计算机所能够支撑的功能不断丰富，从图像处理、视频解码，到三维渲染、可穿戴计算，再到元宇宙、智能大模型，性能发展显著支撑功能创新，计算机越来越重要。

另一方面，性能提高推动了程序设计语言从机器语言、汇编语言到高级语言的发展，使程序员不再为了节省些许硬件资源而煞费苦心。进一步，人工智能等新应用所带来的表达复杂性也推动编程方式不断演进，在强大性能的支持下，编程语言逐步朝着更接近自然语言的方向发展。

Python 则是当代编程语言发展的杰出代表。

拓展： 摩尔定律

摩尔定律（Moore's law）是计算机发展历史上最重要的预测法则，由英特尔（Intel）公司创始人之一戈登·摩尔（Gorden E. Moore）于 1965 年提出。注意，它不是物理或自然法则。摩尔定律预测，单位面积集成电路上可容纳晶体管的数量约每两年翻一倍。由于计算机领域几乎所有的重要部件，如 CPU、内存、接口芯片、网络设备等，都由集成电路实现，摩尔定律实际上揭示了 1965 年至今仍在高速发展的半导体技术趋势。摩尔定律成为计算机性能水平的一个重要预测法则。图 1.1 所示是摩尔定律在 1971 年至 2023 年间的发展趋势，请注意，坐标系纵轴为指数关系。

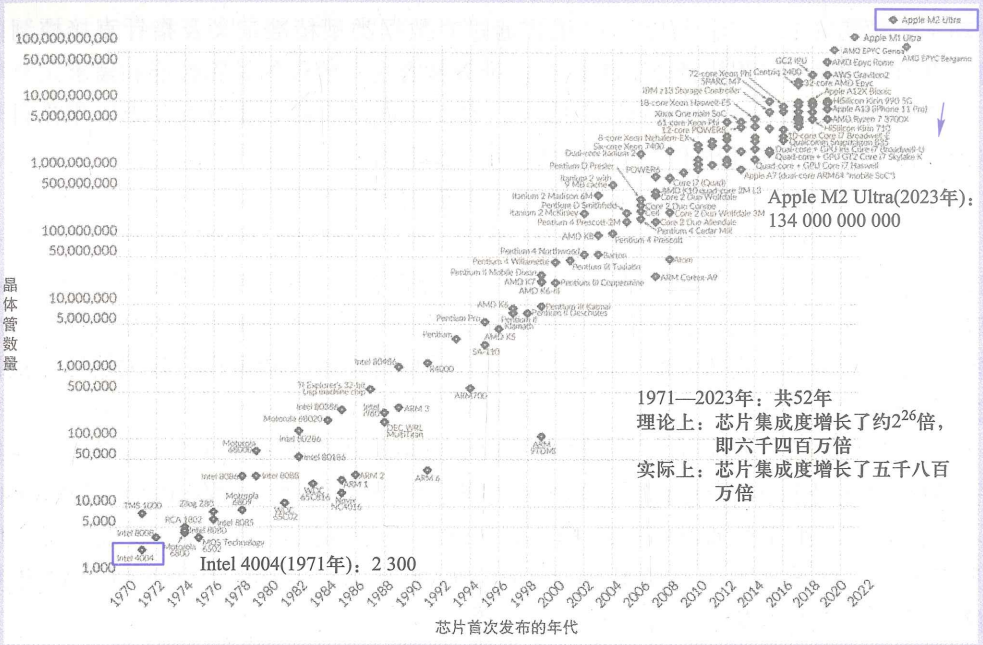


图 1.1　摩尔定律发展：1971—2023 年

如果穿越回 1965 年，计算机还只是科学研究装置，不仅如此，计算器在当时尚未诞生（第一个便携式电子计算器在 1970 年才诞生），计算设备离生活非常遥远。彼时，摩尔定律所预测的指数发展趋势十分惊人，被大多数人认为十分荒诞，因此，它不断被质疑，却一次次被证实。技术发展带来的类似震撼至今仍然“默默且显著地”改变着人类生活，如同十几年前大多数人无法想象手机会运行几百个应用一样。换个角度思考，如果利用技术视角和专业能力去主动审视那些当代重要的预测法则，将能够借助规律“预测未来”，超前布局，更好地迎接每个技术震撼带来的时代变革，如此，将更有可能成为时代创新的引领者。

自 1946 年第一台真正实用的电子数字计算机诞生以来，计算机技术先后经历了多次重大发展变革，与之相适应，计算机在功能性和可编程性两方面的具体表现也不尽相同，

计算机技术整体发展具有鲜明的时代性。

微视频 1-2：计算机技术的演进



计算机技术先后经历了四个主要发展阶段：计算机系统结构阶段、计算机网络与视窗阶段、复杂信息系统阶段、实用人工智能阶段。

第一阶段：1946—1981 年，计算机系统结构阶段。

该阶段始于 1946 年，以全球第一台真正实用的电子数字计算机 ENIAC 诞生为标志，主要服务于科学计算和商业应用。该阶段主要面向科研、政府、关键行业任务需求，探索计算机系统发展的各种结构可能，先后诞生出满足不同需求的超级计算机、高性能计算机、大型机、小型机、工作站、个人计算机等新型计算机设备。

与这个时期有限的计算性能和功能相对应，计算机的可编程性主要表现为合理划分软硬件接口、高效控制计算部件、支撑程序高速计算，程序设计需要在编程逻辑和系统结构之间、处理能力和存储容量之间、计算和通信之间寻找优化和折中。这种精细化的编程需求推动了 C 语言（1972 年）诞生。C 语言通过对数据类型精准定义及指针直接控制内存使用，进而使程序在有限资源下高速运行，十分契合这个阶段的计算性能和需求。

计算机技术的第一个阶段持续了 35 年，主要面向政府或行业重大需求，用作科学计算装置。接下来，计算机技术进入了面向大众的新阶段。

第二阶段：1981—2007 年，计算机网络与视窗阶段。

该阶段始于 1981 年，以 IBM PC 的诞生为标志，几乎同时，1982 年，TCP/IP 协议正式启用，互联网时代正式到来。在这个阶段，计算机技术主要围绕网络技术、视窗技术、多媒体技术发展，以个人计算机和服务器为主要计算平台，提供满足个人需求的视窗应用和网络服务。图形式、视窗式、交互式操作系统不断完善，代替了命令行式的计算机控制方式，成为主流。

由于网络将不同类型系统互联，程序可以更容易地被传播与分享，在多种操作系统上跨平台执行成为计算机编程的迫切需求，由此诞生了具备跨平台功能的 Java 语言（1995 年）。与此同时，由于微软 Windows 操作系统在个人计算机领域的高度普及，视窗应用“所见即所得”的开发需求推动了 Visual C++（VC）、Visual Basic（VB）（1991 年）等视窗编程语言的诞生与流行。

计算机技术的第二个阶段持续了 25 年，让计算机真正走入社会，服务大众需求。接下来，计算机技术进入了面向移动网络应用的新阶段。

第三阶段：2007—2017 年，复杂信息系统阶段。

该阶段始于 2007 年，以美国苹果公司 iPhone 智能手机推出为标志，同时，2008 年，安卓（Andriod）开源移动操作系统发布。在这个时期，一批新的计算概念和技术几乎同时提出并显著推动计算技术的升级换代，这些概念包括但不限于：移动互联网、网格计算、多核众核、云计算、可信计算、边缘计算、可穿戴计算、大数据、物联网、区块链等。这些概念的提出反映了计算应用的多样性和复杂性，以及更严峻的安全问题。虽然概念众多，但没有哪个概念能够引领这个阶段的技术发展，这说明，计算机技术的发展已经进入了复杂信息系统阶段，任何系统都需要不间断地完善才能够提供更加安全可靠的功能。

复杂信息系统之间通过互联网、开源项目和社交关系高度关联，人们逐渐认识到计算机系统的复杂性已经到达人类所能够掌控的边界。面对复杂的计算功能和紧迫的迭代周

期，计算机需要更高抽象级别的程序设计语言来表达可编程性，Python 语言（2008 年 3.0 版本）蛰伏已久并快速发展，成为该阶段最流行的编程语言。

计算机技术的第三个阶段只持续了 10 年，随着深度学习算法的提出及 AlphaGo 轻松超越所有顶尖人类围棋手的卓越表现（2017 年），起起伏伏 60 余年的人工智能再次成为主流，推动计算机技术进入“智能”的新阶段。

第四阶段：2017 年至今，实用人工智能阶段。

该阶段始于 2017 年，以 AlphaGo 卓越表现为标志。当代人工智能已有阶段性进展，能够在诸如人脸识别、机器翻译、自然语言理解、图像搜索等方面成熟应用，进而形成可实用的人工智能技术体系。随之而来，各行业新发展都冠以“智慧”或“智能”字样，形成诸如智慧社会、智慧社区、智慧农业、智慧教育、智慧体育等新名词，似乎不加这些字样就不先进一样，可以看出整个社会对智能技术的热烈期盼。

然而，应该清楚认识到，当下人工智能还处于较低的技术水平，主要靠算力、数据和简单算法支撑，并不具备可解释的逻辑推理能力，只能替代人类解决一些非关键的规程式任务，释放非智力或常规性人类工作。

随着量子计算、概率图理论、大模型、超深度学习、脑机接口等新一代信息技术发展，未来某个时期可能会出现更智慧的人工智能阶段，形成新的智能体。国际人工智能领域顶级专家预测，实现与人类相似的人工智能至少还需要 20 年。彼时，计算机或许已经没有独立载体，它将通过互联网、数据和机器整合一切可用资源，逐步接管人类所有非创造性工作，计算机技术将进入一个未知的新阶段。

纵观计算机技术短暂的发展历史，计算机功能性和可编程性发展存在相互促进的协同关系。一方面，计算机的编程方式逐步让计算机更有效地理解人类意图，进而丰富计算机的功能。另一方面，计算机的功能发展进一步促进编程方式的发展。

或者说，计算机正在借助人类智慧不断“进化”，奔向奇点。

思考与练习 1-1



思考与练习

1.1　计算机的定义是什么？它有哪两个显著特点？

1.2　请调研并阐述不少于 3 个类似摩尔定律的预测法则或评估法则。

1.3　请列出并阐述不少于 5 个近 10 年出现的计算机新技术名词。

1.2　Python 语言概述

微视频 1-3：Python 语言概述



要点： Python 是一个语法简洁、生态丰富、模式多样的通用脚本语言。

1.2.1　程序设计语言

程序设计语言，也叫编程语言，它是计算机理解和识别用户操作意图的一种交互体系。

微视频 1-4：Python 语言的特点



具体来说，编程语言使用计算机指令按照特定规则表达用户意图，使计算机按照设定目标自动运算。这些表达用户意图的计算机指令称为**计算机程序**。由于程序设计语言和编程语言概念一致，本书将交替使用两种说法。

既然是交互方式，为何不能直接使用自然语言呢？很多科幻电影有类似情节，采用人类语言驱动计算机。这种设想非常美好。遗憾的是，尽管许多一流科学家为此做过很多努力，仍然无法在可预见的未来设计出完全理解人类语言的编程工具。

诚然，即使计算机能理解人类语言，人类语言也不适合描述复杂算法，因为人类语言具有不严密和模糊的缺点。例如，“我看见一个人在公园，带着望远镜”，基于常识和经验，交谈双方大多数时候能够理解彼此表达的意思，但深究一下，究竟是“我”带着望远镜，还是“一个人”带着望远镜呢？这种模糊性经常产生错误理解和歧义。相比人类语言，编程语言语法严密，语义表达准确，没有二义性。

程序设计语言包括四类：机器语言、汇编语言、高级语言和生态语言。

机器语言是一种二进制编码语言，使用二进制代码表达指令，由计算机硬件直接识别并执行。例如，执行数字 2 和 3 的加法，16 位计算机上的机器指令为：11010010 00111011，不同 CPU 型号的计算机可能采用不同机器指令。因为二进制程序难以阅读，直接使用机器语言编写程序十分烦冗，因此，科学家发明了汇编语言。

汇编语言使用助记符与机器语言的指令一一对应，在计算机发展早期帮助程序员提高编程效率。例如，执行数字 2 和 3 的加法，汇编语言指令为：“add 2, 3, result”，运算结果写入变量 result。与机器语言类似，不同计算机结构的汇编指令不同。由于机器语言和汇编语言由计算机硬件直接执行，它们统称为低级语言。低级语言有一个显著的缺陷，这些程序只能在特定类型处理器上运行，限制了程序的使用范围。

高级语言帮助程序员更好地表达程序逻辑，更容易地描述计算问题，但与计算机结构无关，采用高级语言编写的程序能够经过翻译在不同计算机上执行。例如，执行数字 2 和 3 的加法，高级语言代码为：“result = 2 + 3”，这个代码只与编程语言有关，与处理器所对应的机器语言无关。随着软件规模的逐步增加，共享优质代码成为提升程序开发效率的重要方式，编程语言再次演进。

生态语言是高级语言的一种演进，除了具有高级语言所代表的编程功能，还能够无缝地支撑优质代码共享和使用，显著缩短开发时间。例如，执行数字 2 和 3 的加法，生态语言代码为：“result = sum(2, 3) ”，其中加法运算由函数 sum() 完成，该函数可能是全球某个程序员编写，而当前程序通过简单调用即可使用。试想，这个“加法运算”可能是人脸识别或某种更复杂算法，程序员可以将其视为黑盒子，仅通过给定参数即可使用，极大缩短程序的开发周期。广义来说，生态语言也是高级语言。

第一个通用性的高级语言是诞生于 1954 年的 FORTRAN 语言。随后的 70 年先后诞生了 600 多种程序设计语言，但是大多数语言由于应用领域的狭窄退出了历史舞台。目前，全球最流行且规模最大的生态语言是 Python 语言。至今还经常使用的程序设计语言有几十种，包括 C、C++、C#、Go、HTML、Java、JavaScript、PHP、Python、SQL、Verilog 等。C 语言和 Python 语言是更通用的编程语言，相比专用于某些领域的编程语言生命力更强。

1.2.2　Python 语言的发展

微视频 1-5：超级语言的诞生



Python 语言诞生于 1991 年，由吉多·范罗苏姆（Guido van Rossum）设计并领导开发，以开源开放为设计理念。

回顾历史，1989 年 12 月，Guido 考虑启动一个开发项目用来打发圣诞节前后的时间，决定将正在构思的新脚本语言开发出来，由此在 1991 年诞生了 Python 语言。该语言以“Python”命名源于 Guido 对一部英剧 *Monty Python's Flying Circus* 的极大兴趣。尽管 Python 语言只因 Guido 大牛“偶然”所思而诞生，但经过 30 余年的发展和应用，已经成为当代计算机技术发展的重要标志之一。历史往往充满偶然，但这样偶然的成功都依赖于英雄式的人物！

Python 语言是开源项目的优秀代表，其解释器代码全部开源开放，可以在 Python 语言主网站自由下载。Python 软件基金会（Python Software Foundation，PSF）拥有 Python 2.1 之后所有版本的版权。PSF 是一个非营利组织，由全球一批顶尖程序员自发组成，致力于更好推进并保护 Python 语言的开放性。

“开源开放”是一个伟大的理念。

形式上，计算机程序可以表示为电磁状态或电磁信号，可以零成本存储和传播，具备共享开放的天然条件。本质上，计算机程序承载了人类智慧，开源就是智慧的展示与传承。基于互联网，开源开放让人类智慧无成本传播，借助程序，这些智慧不仅可以实现具体功能，提供直接服务，还能够传播先进技术理念，推动科技快速发展。科技大爱、造福世界，不过如此！

拓展：开源软件

开源软件（open source software）是一类开放源代码软件的统称，这类软件的源代码在特定许可范围内，可以被任何人学习、修改甚至发布。开源软件从 1998 年开始被众多资深程序员定义并推动，至今，世界上有几百万个开源软件项目，覆盖几乎所有软件应用领域。开源软件通过互联网建立零成本传播方式，降低了学习成本，为计算机技术快速发展提供了极大便利。开源软件也有知识产权要求，由许可协议规定，如 GNU 通用公共许可协议、MIT 许可协议、Apache 许可协议等，一般这些许可协议只对商业使用有限制性要求。如今，全球 80% 以上服务器运行开源软件，每年为用户节省万亿人民币以上的使用成本。开源软件显著推动了当代计算机技术的应用发展，在深层次上影响着未来信息技术的发展速度和普及程度。

涅槃重生，Python 3.x 是现在和未来。

2000 年 10 月，Python 2.0 正式发布，标志着 Python 语言完成自身演进，开启了 Python 广泛应用的新时代。2010 年，Python 2.x 系列发布最后一版，版本号为 2.7，用于终结 2.x 系列版本的发展，不再进行重大改进。

早在 2008 年 12 月，Python 3.0 就正式发布，这个版本解释器采用面向对象方式重构实现，在语法层面做了较多重大改进。所付出的代价是 3.x 系列代码无法向下兼容 2.x 系

列的既有语法，因此，所有基于 Python 2.x 系列版本编写的代码都必须经过修改才能被新版本解释器运行。

Python 语言经历了一个痛苦但令人期待的版本更迭过程：从 2008 年开始，用 Python 编写的几万个标准库和第三方库开始了版本升级过程；历时 8 年至 2016 年，所有重要的 Python 标准库和第三方库都已经在 Python 3.x 版本下进行演进和发展。Python 语言版本升级过程宣告结束。

或许，很多专家都会建议 Python 采用兼容方式进行升级，即 Python 3.x 系列完全兼容 Python 2.x 系列语法，同时推荐使用新语法。然而，对于全球工程师自发构建的开放社区来说，这种渐进式升级方式由于缺少必要驱动将十分缓慢，很难淘汰不适用的过时语法，将带来更长的升级周期。既然已决定升级换代，把过渡交给历史、推动全新未来早日到来才是最自信、最勇敢的决定。Guido 伟大！

“Python 2.x 已经是遗产，Python 3.x 是这个语言的现在和未来”。

1.2.3 Python 语言的特点

Python 语言是一种通用脚本语言，有三个重要特点：语法简洁、生态丰富、模式多样。

第一，语法简洁。Python 语法可以精确表达问题逻辑，更接近自然语言。实现相同程序功能，Python 语言的代码行数仅相当于其他语言的 1/10 至 1/5。更少的代码行数、更简洁的表达方式带来更少的程序错误以及更短的开发周期。

第二，生态丰富。Python 语言提供了几百个内置类和功能库，此外，全球程序员通过开源社区贡献了超过 50 万个第三方库，覆盖几乎所有计算机技术领域。Python 具备良好的编程生态，通过利用已有功能库可以避免重复造轮子，“站在巨人肩膀上”的编程理念能够显著缩短开发周期。

第三，模式多样。总体上，Python 提供了三种编程模式：过程式、对象式和生态式，覆盖了当今编程语言所支持的所有模式。直接编写几行代码，或通过函数组织计算功能，或定义对象封装继承，或将 C/C++ 功能封装成功能库，Python 都能够应对自如。这些编程模式提供了足够的灵活性和创新空间，被广泛使用。

需要注意，有人说 Python 是面向对象编程语言，这是错误的。Python 语法很多元素都包含“属性”和“方法”等面向对象特点，但从编程模式来看，Python 不仅支持面向对象编程，也支持过程式编程，因此，不能认为 Python 是面向对象编程语言。

除了上述三个重要特点外，Python 语言还有一些其他特点。

（1）**平台无关**：Python 程序可以在任何安装解释器的计算机中执行，因此，可以不经修改地实现跨操作系统运行。

（2）**强制可读**：Python 通过强制缩进来体现语句间的逻辑关系，类似文章段落的首行空格，显著提高了程序的可读性，进而增强 Python 程序的可维护性。

（3）**支持中文**：Python 3.x 版本采用 Unicode 编码表示字符信息。Unicode 是一种国际通用的字符编码体系，这使得 Python 程序可以直接处理英文、中文、法文、德文等各类自然语言字符，在处理中文时更加灵活且高效。

（4）**类库便捷**：使用功能库是 Python 语言开发的重要能力。Python 功能库从安装到使用都非常便捷，使用 pip 命令可以用一行代码安装第三方库，使用 import 保留字可以用一行语句引入功能库的具体功能。

（5）**开源理念**：对于高级程序员，Python 语言开源的解释器和功能库具有强大吸引力。更重要地，Python 语言倡导的开源开放理念为信息技术快速发展奠定了坚实的群众基础，语言有文化，世界便不一样了。

本书后续章节将会逐步揭开 Python 语言的神秘面纱，带领读者利用 Python 语言解决一个个有趣又实用的计算问题。后续行文除个别地方外，将用“Python”代替“Python 语言”，并根据行文内容交替使用“编程”和“程序设计”。

思考与练习 1-2



思考与练习

1.4　请列出不少于 10 种编程语言，并简要说明其适用范围。

1.5　请列出不少于 3 个软件开源的社会意义。

1.6　请列出不少于 5 个 Python 语言的特点。

1.3　Python 开发环境配置

微视频 1-6：Python 程序编写与运行



要点： IDLE 是一个轻量级 Python 语言开发环境，可以支持交互式和文件式两种程序运行方式。

1.3.1　开发环境安装

运行 Python 程序需要安装 Python 解释器。

Python 解释器是一个轻量级的小尺寸软件，约为 20 ~ 30 MB，下载网址如下：

https://www.python.org/downloads/

也可以在 Python123 中文网站上下载，网址如下：

https://python123.io/downloads/

Python 主网站下载页面如图 1.2 所示，本书关联的 Python123 下载页面如图 1.3 所示。

无论使用哪个网站，下载 Python 解释器都需要选择相适应的操作系统。如图 1.2 所示，单击图中 **矩形框①** 内的按钮下载 Python 3.11 版本程序。图中位置展示了 Python 最新版本解释器。随着 Python 语言发展，会有更新的版本，但不影响本书内容，读者可以选择之后的新版本。由于 Python 版本更新较快，但很多第三方库更新却较慢，考虑兼容性，建议使用 Python 3.11 版本，不一定使用最新版本。

以 Windows 操作系统为例，单击图 1.2 中的按钮，下载 python-3.11.5-amd64.exe 文件（如果使用其他操作系统，请选择 **矩形框②** 内的相应链接，找到对应文件进行下载）。或者，如图 1.3 所示，在 Python123 网站下载页面上根据操作系统直接选择相应的链接。对于 Windows 操作系统，**矩形框③** 是 32 位的 Python 3.11.5 安装程序，**矩形框④** 是 64 位的安装程序。

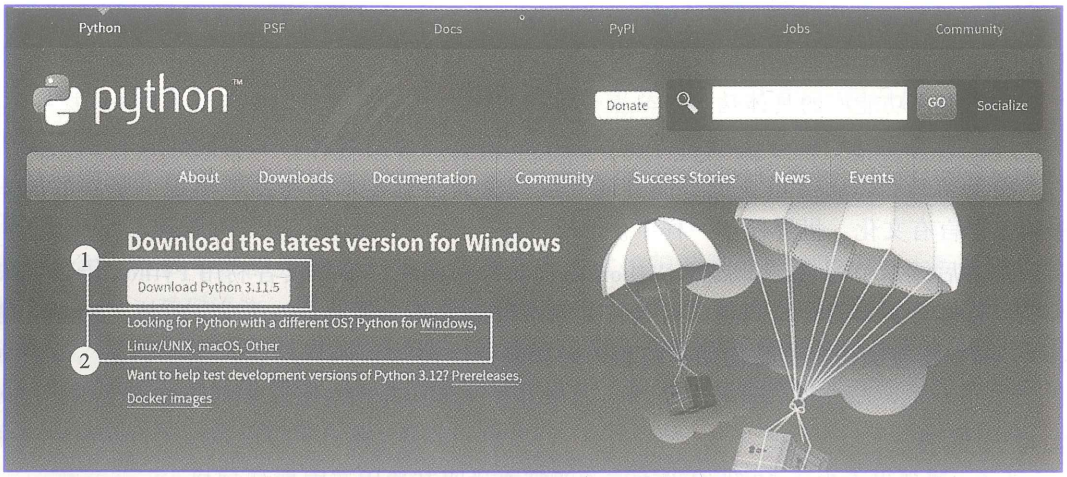


图 1.2　Python 主网站的下载页面



图 1.3　Python123 网站的下载页面

对于初学 Python 的读者，采用 Python 3.5.3 及之后版本、32 位或 64 位均可，不影响编程学习，不一定使用最新版本。部分 Windows 7 及更早操作系统无法安装 3.5.3 及之后版本，请安装并使用 3.4.2 版本。

执行下载后的 Python 安装文件，将产生一个启动页面，如图 1.4 所示。在该页面中，勾选图中 **矩形框⑤** 内的“Add python .exe to PATH”复选框，再执行 **矩形框⑥** 链接启动安装过程。

安装成功后将显示成功界面，如图 1.5 所示。

IDLE 是 Python 编程的主要入门级工具。

除了 Python 解释器外，安装程序将提供一批与 Python 开发和运行相关的工具，最重要的是 Python 集成开发环境 IDLE（Python's Integrated DeveLopment Environment）。启动 IDLE 可以进行 Python 编程，并运行 Python 程序。

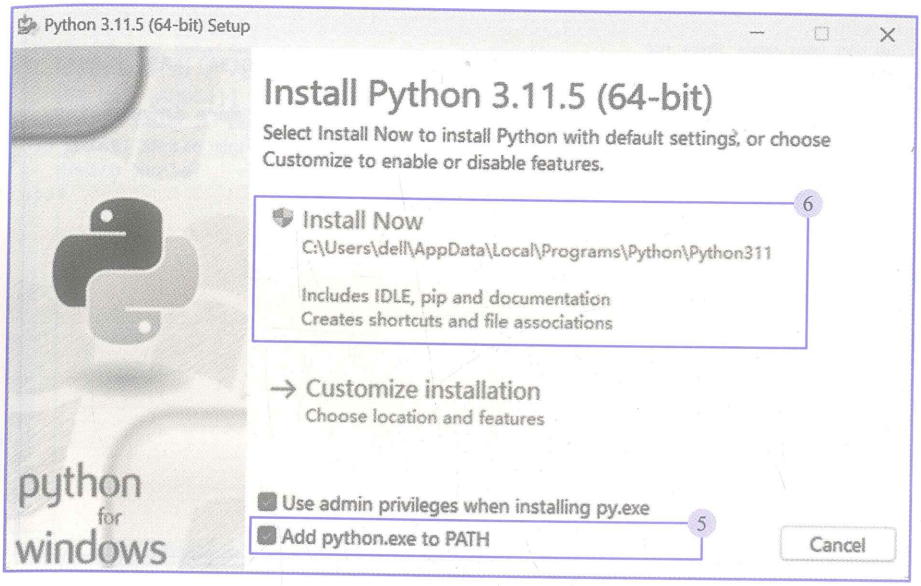


图 1.4　安装程序的启动页面

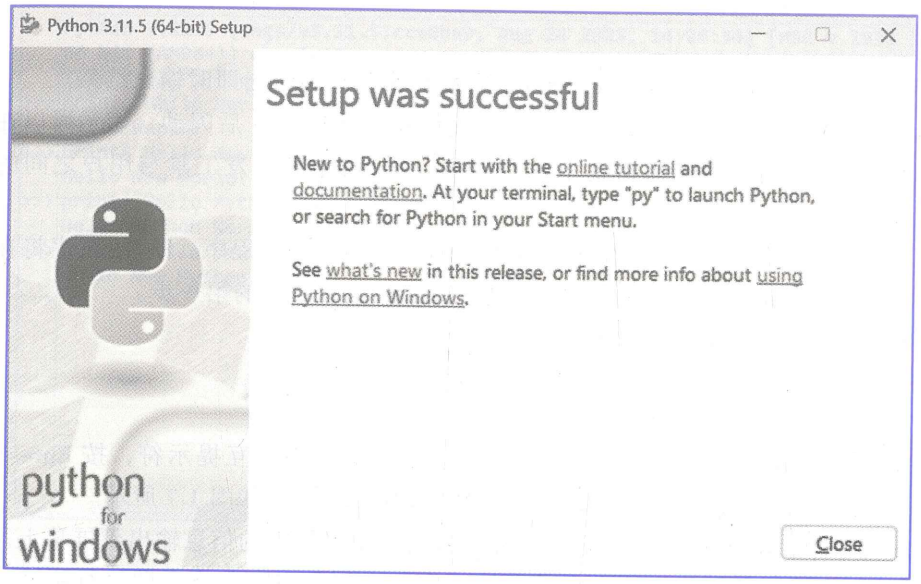


图 1.5　安装程序的成功页面

以 Windows 操作系统为例，成功安装 Python 解释器后，在 Windows “开始” 菜单中搜索关键词 “IDLE”，找到 IDLE 的快捷方式，打开 IDLE 开发环境，如图 1.6 所示。其中，提示符（>>>）之前的信息是 IDLE 开发工具的版本信息，可以忽略。

运行 Python 程序有两种方式：交互式和文件式。

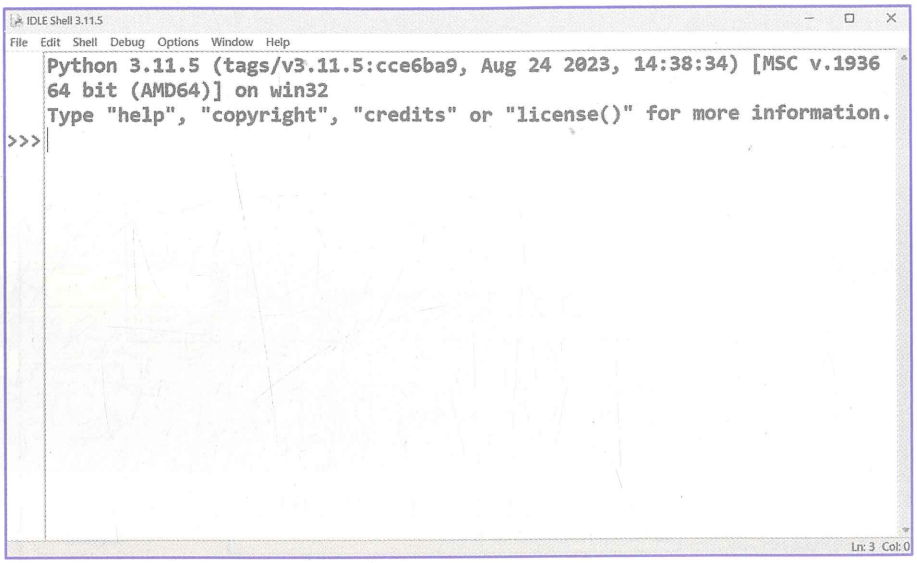


图 1.6 IDLE 开发工具

1.3.2 交互式运行

交互式运行方式利用 IDLE 交互特性，对每行代码输入给出即时输出结果。

IDLE 启动后直接进入交互式运行模式。光标在提示符 >>> 后闪动，表示可以在此处输入 Python 语句，运行后结果将在下一行输出，这是 IDLE 开发工具与用户之间的一种实时交互的运行方式。

学习一门新的编程语言有个惯例：编写并运行最小 Hello 程序。Hello 程序将在屏幕上打印“Hello World!”。程序虽小，却是初学者接触编程语言的第一步。

Python 的 Hello 程序只有一行代码。

如下：

```
print("Hello World!")
```

源代码 1-1：Hello 程序

在 IDLE 交互式环境下中输入上述代码，其中，>>> 为交互提示符，按 Enter 键后，将在下一行得到程序的输出结果，显示为：“Hello World!”，如图 1.7 所示。

Hello 程序中，**print()** 是一个函数，能够将括号中引号内的信息输出到屏幕上。继续输入以下 3 行代码，每行代码输入完成后按一次 Enter 键，IDLE 将立即执行所输入语句并返回结果。交互式运行的结果如图 1.8 所示。

```
print("Hello New World!")
print("Hello Python World!")
print("Hello New Python World!")
```

交互式运行一般用于代码的运算或调试，不是常规的编程方式，仅适合测试少量 Python 语句的运行效果。关闭窗口将退出 IDLE 运行环境，亦可在提示符 >>> 后输入 exit() 或 quit() 退出 IDLE。

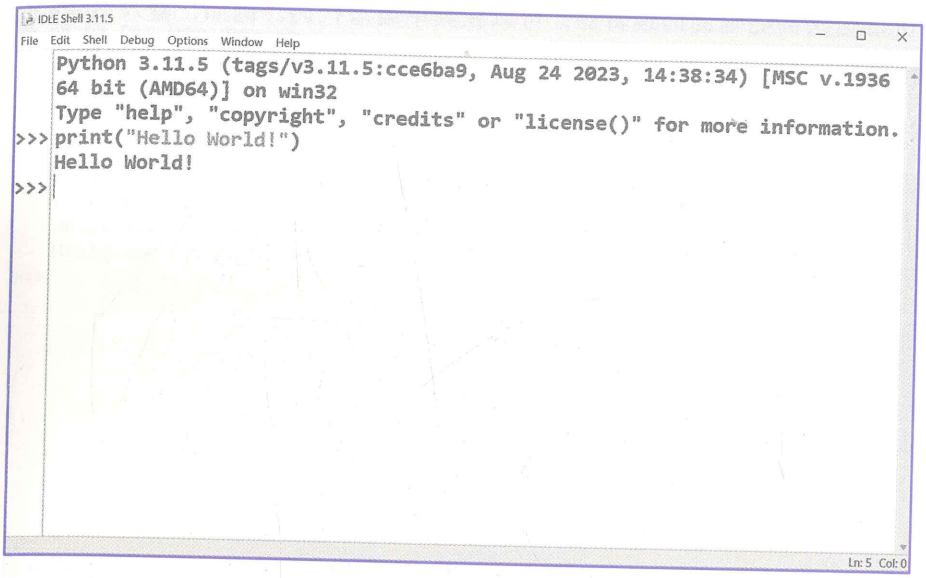


图 1.7　Hello 程序的交互式效果

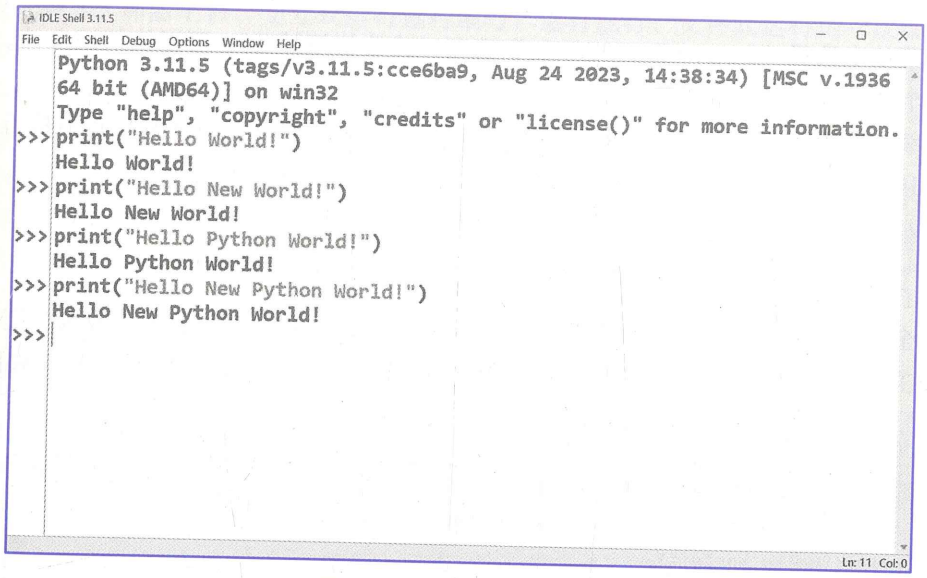


图 1.8　交互式运行的效果

1.3.3　文件式运行

文件式运行方式将多行 Python 程序一次性写入文件中，然后启动 Python 解释器批量执行文件中的全部代码，这是最常用的程序运行方式。

文件式运行在编写代码时，可以用 IDLE，也可以用其他文本编辑工具或专业编程工具，在程序运行前，程序文件只是一个文本文件。

IDLE 提供文件式运行的支持。IDLE 启动后，在菜单中选择 **“File → New File”** 选项，或使用快捷键 Ctrl+N 打开一个新的**编辑窗口**。这个新窗口不是交互模式，它是一个

具备 Python 语法高亮辅助的编辑器，可以在其中编写代码。例如，输入 Hello 程序并将文件保存为 hello.py，如图 1.9 所示。

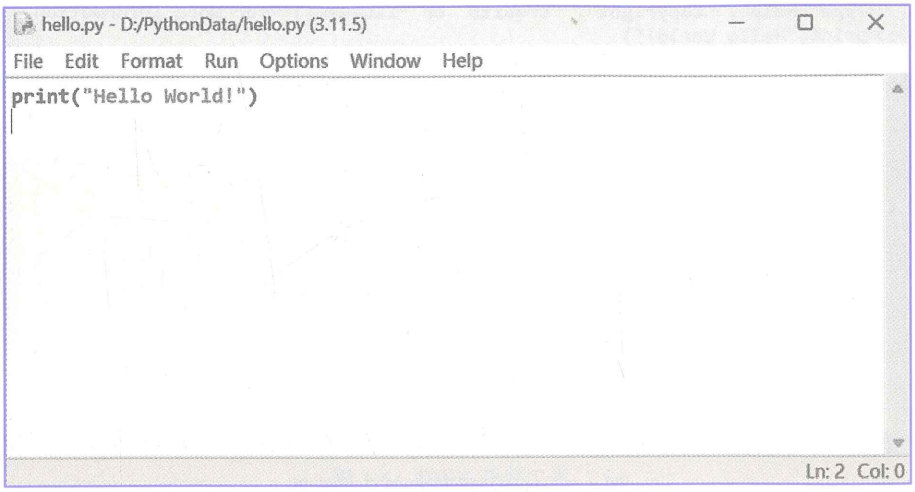


图 1.9 Hello 程序的文件式运行的代码编写

在菜单中选择“Run → Run Module”选项，或按快捷键 F5，可以执行该文件。IDLE 启动一个交互环境显示输出结果。

继续在 hello.py 文件中输入以下 3 行代码，在输入每行代码时 IDLE 并不执行程序，如图 1.10 所示。

```
print("Hello New World!")
print("Hello Python World!")
print("Hello New Python World!")
```

当全部代码输入完成后，在菜单中选择“Run → Run Module”选项，或按快捷键 F5，执行该文件。IDLE 启动一个交互环境，显示文件中代码运行后的输出结果，如图 1.11 所示。

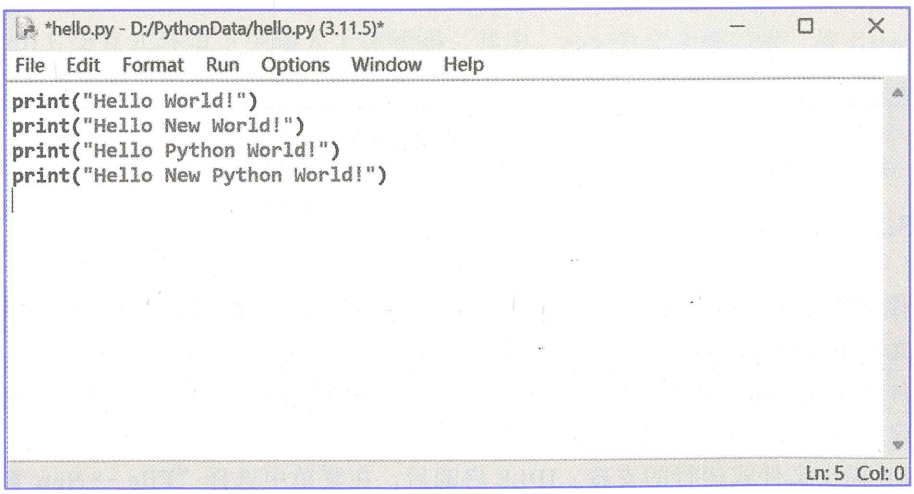


图 1.10 文件式运行的代码编写

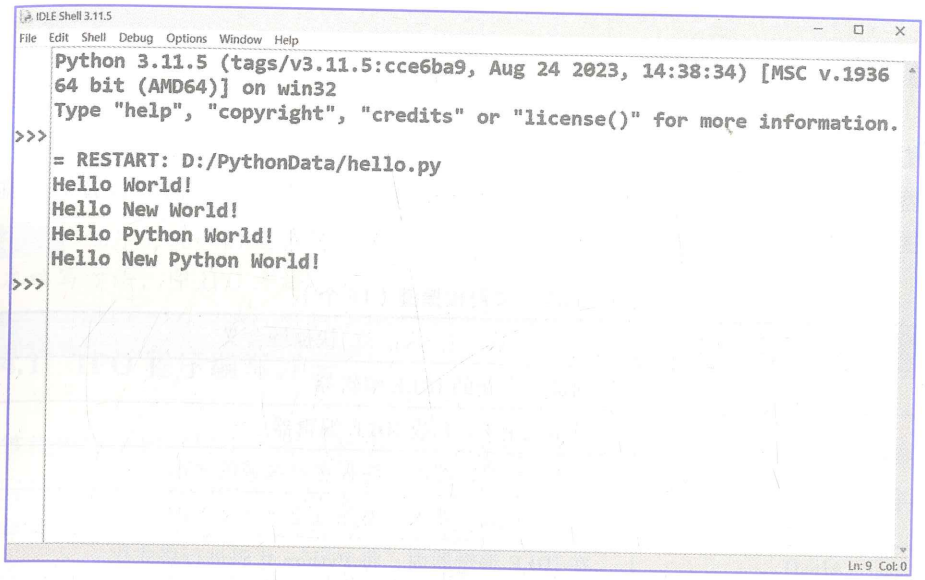


图 1.11　文件式运行的效果

对比图 1.8 和图 1.11，交互式运行逐行编写、逐行执行，文件式运行则一次编写一次全部执行。文件式运行更体现程序设计的正常过程。实际上，Python 代码可以在任意编辑器中编写，编写完成之后可以用 IDLE 打开并执行。

IDLE 是小规模 Python 软件项目的主要编写工具。

IDLE 是一个简单有效的集成开发环境，无论交互式或文件式，它都有助于快速编写和调试代码。对于 100 ~ 300 行左右代码的编程项目，建议使用 IDLE 编辑器，更大规模代码可以使用更加专业的开发工具，如 Visual Studio Code、PyCharm 或 Jupyter Notebook。

拓展：Jupyter Notebook

Jupyter Notebook 是一个比 IDLE 更高级的 Python 编辑器，它能够“所见即所得”地编写、运行和维护 Python 程序，主要用于科学计算、数据分析与可视化、机器学习等领域，可以在 https://jupyter.org/ 下载并安装。

Jupyter Notebook 能够灵活整合大量第三方库功能，体现 Python 在数据科学与人工智能领域的优势。然而，该工具主要服务于专业应用开发，并不适合 Python 初学者。建议完成 Python 语言入门后，再学习使用该工具，将事半功倍！

什么是最好的 Python 编程工具？对于大多数使用者，IDLE 和 Jupyter Notebook 就是最好的 Python 工具组合。IDLE 体现了 Python 编程本源，简单实用且十分灵活；Jupyter Notebook 则更适合数据分析与处理。编者将在后续书籍中讲解 Jupyter Notebook 使用方法，帮助学习者更酷、更实用、更专业地玩转数据，具备数据全周期处理能力，掌握智能时代的数据分析方法。对于初学者，此刻，还请用好 IDLE。

1.3.4 IDLE 快捷操作

IDLE 快捷键可以代替鼠标快速完成某些特定操作，让编程更加高效和愉快。

键盘的上下左右箭头可以在 IDLE 中移动光标，以便确定代码输入位置。其他 IDLE 常用快捷键如表 1.1 所示。

表 1.1 IDLE 常用快捷键（16 个）

快捷键表示	快捷键含义
Ctrl+N	创建一个新的 IDLE 编辑器
Ctrl+Q	退出当前 IDLE 或 IDLE 编辑器
Alt+3	在 IDLE 编辑器内，注释选定区域的文本
Alt+4	在 IDLE 编辑器内，解除选定区域文本的注释
Alt+Q	在 IDLE 编辑器内，将 Python 代码进行格式化
F5	执行当前 Python 程序
Ctrl+Backspace	快速删除光标左侧内容，每次删除一个词
Ctrl+Del	快速删除光标右侧内容，每次删除一个词
Ctrl+ 上下左右箭头	快速向上下左右方向移动光标
PageUp	快速向上移动光标，每次移动 10 行
PageDown	快速向下移动光标，每次移动 10 行
Home	光标移动至当前行的开头
End	光标移动至当前行的结尾
Ctrl+Home	光标移动至文件的开头
Ctrl+End	光标移动至文件的结尾
Ctrl+K	删除光标当前行之后的内容

本书所有程序都可以通过 IDLE 编写并运行。行文方面，对于单行代码或需要观察输出结果的情况，本书采用 IDLE 交互式（由 >>> 引导）进行描述；对于整段代码讲解的情况，采用文件式进行描述。

对于能够正确运行的程序，总是希望能够通过单击鼠标直接运行，并隐去 Python 源代码，这个过程叫作“程序发布”，本书将在 8.7 节中介绍利用 pyinstaller 库在 Windows 等操作系统中发布程序的方法，让程序运行更酷！

思考与练习 1-3



思考与练习

1.7 请采用交互式方法在 IDLE 中输出“祖国，你好！”。

1.8 请列出不少于 3 个 IDLE 快捷键及其功能。

1.9 请列出不少于 3 个 Python 编程工具。

1.4　程序编写和问题求解

微视频 1-7：程序的基本编写方法



要点： 程序有统一的运算模式，即输入、处理和输出，这种朴素运算模式形成了程序的基本编写方法，即 IPO 方法。

1.4.1　IPO 程序编写方法

计算机程序是用来解决计算问题的。较大规模的程序提供丰富的计算功能，解决完整的计算问题，例如，控制航天飞机运行的指控程序、操作系统等；小型程序或程序片段为其他程序提供特定计算支持，作为解决更大计算问题的组成部分。

无论程序规模如何，程序有统一的运算模式，形成了基本的程序编写方法：IPO 方法。

这种统一的运算模式指输入数据、处理数据和输出结果，分别对应：输入、处理、输出三个过程，简称：IPO。

输入（input）是一个程序的开始。

程序要处理的数据可以有多种来源，因此形成了多种输入方式，包括：文件输入、网络输入、交互输入、随机数据输入、内部参数输入等。

（1）文件输入：将文件作为程序的输入来源。例如，若要对 Excel 文件中的数据进行计算，首先打开文件，根据 Excel 文件格式获得所需数据，进而开展计算。

（2）网络输入：将互联网上的数据作为输入来源。例如，利用网络爬虫捕获互联网数据，根据 HTML 格式进行解析。第 10 章将介绍网络爬虫的基本原理和方法。

（3）交互输入：将程序使用者输入的信息作为输入来源。当程序与用户间存在交互时，程序通过交互界面提示用户输入数据。

（4）随机数据输入：将随机数作为程序输入，这需要使用特定的随机数生成器或调用相关函数。第 4 章将介绍随机数的使用方法。

（5）内部参数输入：程序内部的初始化变量也是一种输入，尽管程序看似没有从外部获得输入，但初始化数据会影响程序运行，因此对程序的功能来说也是输入。

处理（process）是程序根据输入数据产生输出结果的过程。

根据用户定义的运算逻辑对输入数据进行操作，产生输出结果。其中，运算逻辑也被称为**算法**，它是程序中最重要的组成部分。算法是一个程序的灵魂，体现计算智慧。第 1.5 节将介绍算法与设计算法的思维。

输出（output）是程序展示运算结果的方式。

程序的输出方式包括交互输出、图形输出、文件输出、网络输出、变量输出等。

（1）交互输出：以交互界面为输出目标，通过程序运行所提供的交互界面展示运算结果。例如，IDLE 交互界面是 Python 程序结果的一种输出途径。

（2）图形输出：启动独立的图形输出窗口，根据指令绘制运算结果，这是一种高级的人机交互输出方法。

（3）文件输出：通过生成新的文件或修改已有文件方式输出运行结果，这是程序常用的输出方式。第 7 章将介绍文件的使用。

（4）网络输出：通过网络接口输出数据。第 10 章将介绍向搜索引擎自动提交关键词的应用实例。

（5）变量输出：程序将运行结果输出到系统内部变量中，这类系统内部变量能够辅助信息在多程序间传递，包括管道、线程、信号量等。

IPO 不仅是程序编写方法，也是描述计算问题的方式。

以圆面积的计算为例，该问题的 IPO 描述如下：

> **输入：圆半径的值，记为变量 `radius`**
> **处理：计算圆面积，公式为 `area = π*radius*radius`，此处，π 取 `3.1415`**
> **输出：圆面积的值，即变量 `area` 的值**

计算问题的 IPO 描述能够有效界定计算的输入、输出和求解方式，更容易明确各部分需求，一般采用自然语言方式表达。为了区分其他描述方式，本书中所有 IPO 描述都包括“输入”“处理”和“输出”3 个引导词。

IPO 方法可以帮助学习者建立最基本的程序设计概念。虽然它是入门级的程序设计方法，但却是所有高级编程方法的基础。本书第 8 章将进一步拓展，介绍一批实用且有深度的高级编程方法。

1.4.2　问题的求解步骤

编写程序的目的是“利用计算机解决问题”，共包括 6 个步骤：分析问题、划分边界、设计算法、编写程序、调试测试和升级维护。

以上 6 个步骤可以分为两个阶段：程序设计阶段和程序实现阶段，如图 1.12 所示。其中，程序设计阶段覆盖前 3 个步骤，程序实现阶段覆盖后 3 个步骤。

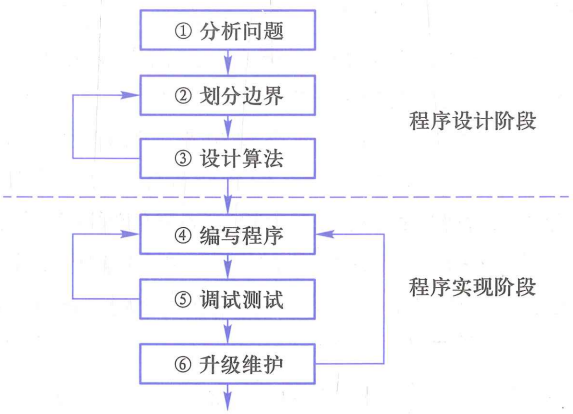


图 1.12　求解问题的两个阶段和 6 个步骤

（1）分析问题：分析问题的计算部分。

首先必须明确，计算机可以解决计算问题，即解决一个问题的计算部分。清楚理解所需解决问题的计算部分十分重要，这是利用计算机解决问题的前提，对计算部分的不同理

解会产生不一样的程序。

例如，本书每章节后都有若干道程序练习题，如果本书被当作教材，教师会根据这些习题布置课后作业。此时，一些读者可能会思考这样的问题：如何由计算机辅助求解习题并完成作业？对这个问题的理解可以有多个角度。

第一，对于作业中的数学计算，可以编写程序辅助完成，但利用哪些计算公式则由读者自己选择或设计。此时，该问题的计算部分表现为对某些数学公式的计算。

第二，可以利用互联网搜索课后练习题的答案，根据搜索结果完成作业。为了降低网络答案错误的风险，可以让计算机获取多份不同来源的答案并选择相同结果最多的答案作为“正确”答案。此时，该问题的计算部分表现为在网络上自动获取多份结果并输出最可能正确结果的过程。

第三，计算机是否可以直接理解课后练习题并给出答案呢？从这个角度来看，该问题的计算部分则表现为计算机对练习题的理解和对人工智能的求解。直到今天，具有高度智能的计算机仍然是全球科学家共同研究的目标。如果确定从这个角度来分析计算问题，读者恐怕无法在短时间内提交作业了。

这个例子说明，对问题的不同理解将产生不同的计算方式，也将产生不同功能和复杂度的程序。如何更好地理解一个问题的计算部分，如何有效地利用计算机解决问题，这不只是编写程序的问题，更是重要的思维问题，即计算思维。

拓展：人工智能和图灵测试

人工智能（artificial intelligence，AI）是计算机科学的一个分支，区别于人类智能，人工智能指由机器或软件所体现的智能。“智能”这个概念难以确切定义，因此，1950 年，计算机科学之父艾伦·图灵（Alan Turing）提出了著名的“图灵测试”，从测试角度描述智能的含义。在图灵测试中，机器和人分别通过文本途径（避免计算机理解语言能力不足的影响）回答一组由独立评判人提出的问题，如果评判人无法从回答中区分机器和人，则认为机器通过测试，具备与人相当的智能。图灵测试并不评判问题答案的正确性，而是通过评判答案之间的相似性确定机器是否具备智能。图灵测试是人工智能中的重要概念，也成为人工智能发展的重要评判基础。

（2）划分边界：划分问题的功能边界。

计算机只能完成确定性的计算功能，因此，需要精确定义或描述问题的功能边界，即明确问题的输入、输出和对处理的要求。可以利用 IPO 方法辅助分析问题的计算部分，给出问题的 IPO 描述，划分功能边界。这个步骤只关心输入、输出以及输入输出之间的总体功能关系，此时先不用关注功能的具体实现方法，找准边界是这个步骤的主要目标。

（3）设计算法：设计问题的求解算法。

在明确处理功能的基础上，如何实现程序功能呢？这需要设计问题的求解算法。简单程序功能的输入和输出关系比较直观，程序结构比较简单，按照步骤直接选择或设计算法即可。对于复杂的程序功能，需要利用程序设计方法将“大功能”划分成“小功能”，或者将功能中相对独立的部分封装成模块，并在各功能或模块之间设计处理流程。对于

“小功能”或模块中的操作，可以将它们看成一个新的计算问题，再回到步骤 2 进行小功能的边界划分，逐级划分问题、逐级设计算法，进而完善一个大功能。更多有关程序设计方法的内容将在第 8 章介绍。

（4）编写程序：编写问题的计算程序。

选择一门编程语言，如 Python 语言，完成程序设计结构和算法的功能编写。原则上，任何通用编程语言都可以用来解决任何计算问题，在正确性上没有区别。然而，不同编程语言在程序的运行性能、可读性、可维护性、开发周期和调试难度等方面有很大不同。Python 程序相比 C 程序在运行性能上略有逊色，不适合性能要求十分苛刻的特殊计算任务；但 Python 程序在可读性、可维护性和开发周期等方面相比 C 程序有较大优势。当代计算机经过长期发展，性能远超过一般功能程序的使用需求，Python 程序在运行性能方面的微弱劣势对解决一般问题并不重要。

（5）调试测试：调试和测试程序。

运行程序，通过单元测试与集成测试等方法评估程序结果的正确性。一般来说，程序错误（俗称 bug）的数量与程序规模成正比。即使经验丰富的程序员编写的程序也会存在 bug，不同只在于 bug 数量的多少和发现的难易。为此，找到并排除程序错误十分必要，这个过程称为调试（俗称 debug）。

当程序正确运行后，可以进行更多测试发现程序在各种情况下的执行特点。例如，压力测试能够获得程序运行速度的最大值和稳定运行的性能边界；安全测试能够发现程序漏洞，界定程序的安全边界，进而指导程序在合理范围内使用。

调试和测试是一对相关但不同的概念，调试用于保证程序按照设计意图正确运行，测试用于分析程序的执行特点，为程序确定可实用的场景。

（6）升级维护：适应问题的升级维护。

任何一个程序都有它的历史使命，在这个使命结束之前，随着功能需求、计算需求和应用需求的不断变化，程序将不断地升级维护，以适应这些变化。要记住，程序是为求解问题服务的，进一步，程序具有特定目的和服务对象，当需求变化时，程序也应该进行调整。一般来说，升级维护仅响应程序使用者的微小需求及功能变化，并回到步骤 4 开始新一轮迭代。如果需求变动很大，需要重新分析问题，则可视为一次新的程序开发。

综上所述，解决计算问题包括 6 个步骤：分析问题、划分边界、设计算法、编写程序、调试测试和升级维护。

与编程语言直接相关的步骤在程序实现阶段，即第 4 到 6 步。

可见，在问题求解过程中，编写程序只是一个环节，在此之前，分析问题、划分边界和设计算法都是重要步骤，也是程序编写的关键。经过 1、2 和 3 这些设计步骤，计算问题已经有了具体的解决方案，这是计算思维的创造过程。步骤 4 和 5 则是对解决方案的计算机实现。因此，在正式场合，我们常用“程序设计”而非“编程”表达计算机求解的过程。这也是本书命名为《Python 语言程序设计基础》的原因，任何以“编程”为名字的描述都不够正式或严谨。

1.4.3　问题的可计算性

——世间所有问题都可以计算吗？

——不是！

计算机不是万能灵药，无法解决所有世间问题。例如，计算机无法求解哥德巴赫猜想，也无法回答诸如“中午该吃京酱肉丝还是鱼香肉丝”的问题，更不能理性回答“谁是世界上最美的女人”这类魔镜问题。

一个问题能否通过计算机来解决的特性，称为问题的可计算性。

计算理论是计算机科学的理论基础之一，用于回答什么是容易计算、什么是很难计算的问题，以及具体阐述可计算性、计算代价、计算难易等问题。抛开深奥理论，计算机的执行过程是算法的运算过程。

一切不能用有限、确定步骤描述的算法都是不可计算的。

例如，确定一个未知数学命题的真或假，这是不可计算的，因为未知数学命题的证明没有确定方法，计算机无法运行未知算法。

简单说，可计算性要求问题具有明确的求解算法，且算法是计算机可执行的。但随着人工智能和大数据等新技术的发展，计算机似乎也能解决一些不那么“明确”的问题。例如，购物网站根据用户过往消费行为，推荐用户可能关心的商品，体现了一定的“不确定性”；智能手环采集用户日常行为，诊断未来可能产生的各类疾病风险。这类“不确定”同样建立在明确算法基础上，“推荐”或“辅助”等功能仅具有概率意义，对个体而言，不体现具体的因果关系。所以，即使有了人工智能，计算机也不能解决世间所有问题。

思考与练习

1.10　针对计算问题：测试一台机器是否真正拥有人类的智能。请用 IPO 方法描述该问题的解决方案。（参考图灵测试）

1.11　在计算机解决问题的过程中，哪些步骤可能用到 Python 语言？

1.12　计算机程序不能解决所有问题。例如，计算机无法回答下列问题：你最欣赏的历史人物是谁？孙红雷和姚晨两位演员，谁的演技更好？创新对我国未来经济的价值有多大？请总结，哪些类型的问题无法通过计算机程序解决？

思考与练习 1-4



1.5　算法和计算思维

要点： 计算思维是人类科学思维活动的重要组成部分，与实证思维和逻辑思维同等重要。

1.5.1 算法概述

算法是完成特定计算的一组有限指令序列，用于描述计算流程。

算法（algorithm）是数学和计算机领域的一个专用词汇，指以有限指令序列的形式，表达完成特定功能的计算方法。大多数算法并没有精巧的构造，而是对计算流程或步骤的细致描述。简单说，算法是方法，用指令序列表达。

对算法的理解有广义和狭义之分。广义上，任何完成计算功能的一组操作都是算法，这组操作可以对应单一的计算问题，也可以对应多个计算问题的组合。狭义上，算法通常针对单一计算问题，例如，针对优化求解问题的贪婪算法、遍历树结构的深度优先算法等。

算法和程序是两个相关但不同的概念。算法指计算所采用的方法，亦称为计算流程或计算方法。对于特定问题，其解决方法可能存在多种可能，即多个算法，但每个算法都是明确的，与编程语言无关。程序指算法的实现，与编程语言有关。算法可以由多种编程语言实现，但受限于编程语言模式和语法，不同编程语言对算法的实现细节不同。

算法通常采用伪代码、流程图和程序等方法描述。

伪代码是采用自然语言方式描述算法的方式。伪代码与具体编程语言无关，因此，算法运行过程的描述可宽泛、可具体，不拘一格。伪代码方法可以借助 IPO 界定输入输出，重点阐述如何在处理（process）中将输入（input）转换为输出（output）。伪代码灵活自然，但受限于使用者的自然语言描述精度，容易出现描述上的二义性。

例如，计算正整数 1 到 R 之间所有整数的算术和，采用循环累加方法求解。该算法的伪代码描述如下：

```
输入：正整数 R
处理：定义变量 S，初始值为 0
      从整数 1 开始至整数 R，将数字逐一累加到变量 S 中
      即：S=1+2+3+…+R
输出：变量 S
```

流程图是一种采用系列图形、线条和文字描述算法的方式。

程序流程图的基本元素包括 7 种，如图 1.13 所示。

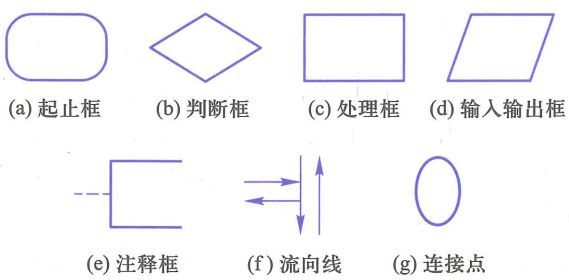


图 1.13 程序流程图的 7 种元素

其中，起止框表示算法的开始和结束；判断框判断一个条件是否成立，并根据判断结果选择不同的执行路径；处理框表示一组处理过程；输入输出框表示数据输入或结果输出；注释框用于增加文本解释；流向线以带箭头直线或曲线形式指示方法的执行路径；连接点将多个流程图连接到一起，用于将一个较大流程图分隔为若干部分。

例如，计算正整数 1 到 R 之间所有整数的算术和，采用循环累加方法求解。该算法的流程图如图 1.14 所示。

流程图描述是算法最直观易懂的表达方式，但仅适用于较短的算法。优点在于直观、清晰且逻辑确定；但缺点也很明显，即流程图绘制比较烦琐，当程序较大时流程图会很复杂，反而降低了表达的清晰性。

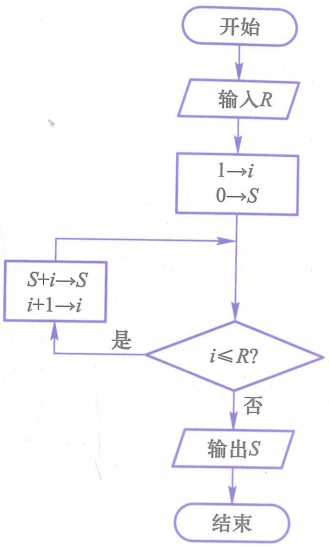


图 1.14　累加算法的流程图

1.5.2　控制结构

算法总体上是顺序执行的一个过程。顺序结构是算法的基础，但单一的顺序结构不可能解决所有问题，因此，需要引入控制结构来更改流程的执行顺序以满足多样化的功能需求。

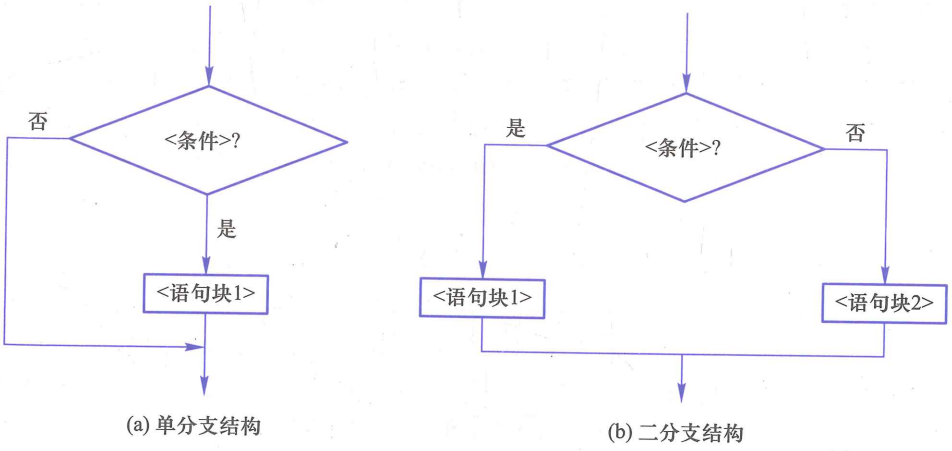
算法由 3 种基本控制结构组成：顺序结构、分支结构和循环结构。

这些控制结构都有一个入口和一个出口。任何算法及程序都由这 3 种基本控制结构组合而成。顺序结构是程序按照线性顺序依次执行的一种运行方式，如图 1.15 所示，其中语句块 1 和语句块 2 表示一个或一组顺序执行的语句。



图 1.15　顺序结构的流程图表示

分支结构是程序根据条件判断结果而选择不同向前执行路径的一种运行方式，如图 1.16 所示。根据分支路径的完备性，分支结构包括单分支结构（图 1.16（a）所示）和二分支结构（图 1.16（b）所示）。将多个二分支结构组合可以形成多分支结构。



(a) 单分支结构　　(b) 二分支结构

图 1.16　分支结构的流程图表示

循环结构是程序根据条件判断结果确定是否返回来路再执行，并可能反复执行的一种运行方式，如图 1.17 所示。根据循环体触发条件不同，循环结构包括条件循环和遍历循环。

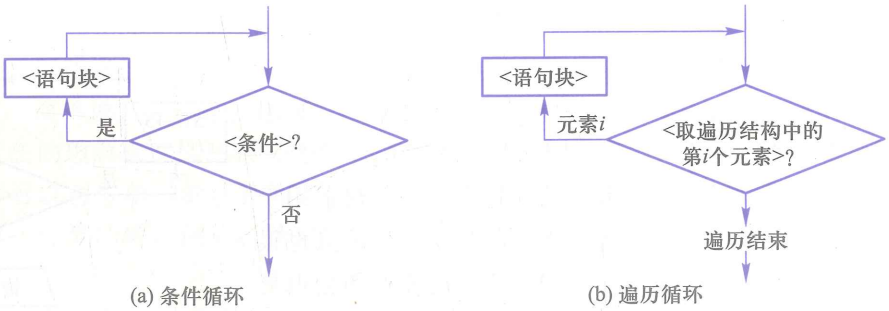


图 1.17　循环结构的流程图表示

Python 语言提供了丰富的语法来支持以上 3 种结构，将在第 4 章详细介绍。

顺序、分支、循环是表达计算过程的基本方式，也是描述算法的基本单元。通过它们的“巧妙”组合，形成一个又一个算法，进而产生了一种“描述流程、抽象计算、基于计算机求解”的思维方式：计算思维。

1.5.3　计算思维

计算思维是依托计算机强大算力演化形成的思维模式。

计算机强大算力带给人类社会震撼，也促进人类思维模式的转变。众多科学家、思想家、教育家都有过这方面论述。2006 年，时任美国卡内基 - 梅隆大学计算机系主任的周以真（Jeannette M. Wing）教授提出了计算思维（computational thinking）的概念。计算思维是运用计算机科学的基础概念进行问题求解、系统设计的一种思维活动。

计算思维并非天生就存在且清晰，人类探索自然几千年中仅有十分“朦胧”的计算概念。即使第一台真正实用的通用电子数字计算机 ENIAC 诞生并带来强大算力，也并没有让人类对计算有新的认识，更难从思维角度理解和认识计算。然而，计算机技术的快速发展改变了人类认知。摩尔定律和网络技术在短时间内让计算机低成本走入人类日常生活，大量传统行业通过信息化改造大幅提升效率，一系列显著变化让人类意识到计算机的强大力量。人类社会已经离不开计算机带来的丰富且强大的算力，这个过程中，人类思维也发生了重要演进。可以看出，计算思维是计算机科学发展到一定程度而提出的，它是人类逐渐意识到计算机强大后自然产生的思维模式。

拓展：ENIAC

ENIAC（Electronic Numerical Integrator and Computer，电子数字积分计算机）是世界上第一台真正实用的通用电子数字计算机，于 1946 年诞生于美国宾夕法尼亚大学摩尔实验室。其实，ENIAC 并非世界上第一台计算机，但却是第一台实用的通用计算机。

ENIAC 的设计思想来源于 1935—1939 年间由美国艾奥瓦州立大学副教授阿塔纳索夫（John Vincent Atanasoff）和研究生贝瑞（Clifford Berry）共同研制的计算机样机 ABC，很可惜，ABC 仅有样机，未能真正使用起来，ENIAC 可以看作是计算机样机 ABC 设计思想的延续。

尽管 ENIAC 占地面积约 170 m^2，重达 30 t，但它的速度比当时最快的计算设备快 1 000 倍。这样惊人的计算性能在随后半个多世纪改变了整个世界，也让我们看到原始创新极其重要。

原始创新的产物看起来总是很“笨拙”、很“原始”，不那么“高大上”，无法与精心包装的已有工业技术比拼颜值。然而，原始创新标志着一种新力量的诞生，如 ENIAC 的计算能力、TCP/IP 的互联能力，或是当前认知难以理解的某个形式。有内生动力的新力量不容小觑，它既可以挑战规则，也能摧枯拉朽，更是人类发展的重要支撑。原始创新很重要却很难获得，需要研究者的智慧、眼光和自信，更需要日积月累地持续学习、试错与思考。伟大的成就，需要伟大的实践！

实证思维、逻辑思维、计算思维是人类 3 种重要的思维方式。

人类在认识世界、改造世界过程中表现出 3 种基本的思维特征：以实验和验证为特征的实证思维，如物理学，提出假设并通过实验进行验证；以推理和演绎为特征的逻辑思维，如数学，建立符号体系并通过推导产生新的结论；以设计和构造为特征的计算思维，如计算机科学，通过计算机强大算力求解问题。

计算思维已经成为人类科学思维活动的重要组成部分。计算思维是人类善于利用工具的重要表现。计算思维并非自古有之，长久以来，人们认为的“计算”是数学，即通过数学表达及推导来解决问题。例如，求解整数 1 到 R 之间所有整数算术和的实例，没有计算机时，主要通过等差数列求和公式进行“公式求解”。若利用计算机，则不需要了解数学公式，采用循环累加的“累加求和”方法即可求解。

人类长久形成的方法论均以人类智慧和能力为边界，因此，“公式求和”比“累加求和”更省事，节省大量人力物力，数学方法更优。然而，计算机诞生了，它代表先进生产力，在计算机强大算力面前，“累加求和”则更容易表达且计算更快，无须枉费人的智力进行“公式求和”。这给人类解决问题带来新的思路，当能够确定性表达计算过程时，计算机可以不惧烦琐、更快地解决问题，进而降低人类表达问题的难度，形成新的思维方法。

计算思维的本质是抽象（abstraction）和自动化（automation）。

计算思维求解问题有如下步骤。首先，思考该问题是否“可被计算”；其次，通过约简、嵌入、转化、分解等方法抽象、拆解并描述问题，将大问题转化为多个小问题；进一步，清晰描述每个问题的解决步骤，无论步骤多么复杂，只要存在结束（停机）条件，就可以形成算法，完成“抽象”过程；最后，用编程语言将抽象后的算法转变为程序，利用计算机进行自动化求解。计算思维的本质是抽象和自动化，即抽象实际问题的计算步骤，利用计算机自动化求解。

计算思维的核心是基于计算机模型的问题约束求解。具体而言，计算思维包含诸多内

涵，比如，近似性、随机化、有限性、递归、采样、冗余、压缩、仿真、并行、分解、容错、纠错、恢复、规划、学习、调度、折中、优化等。计算思维的关键在于“构造”，主要指构造计算问题的求解过程。计算思维并不是让人类像机器一样思考，也不是让计算机像人类一样思考，而是用计算机实现人类抽象的模型或步骤，帮助人类求解问题，它是人类思维中“抽象、构造”思考的集中体现。

1.5.4 计算方法：流程表达法

流程表达法是一种根据解决问题的步骤实现计算过程的方法，具有普适性。

计算思维的抽象依托算法或控制结构来表达，形成问题的求解步骤或流程。进一步，根据流程逐步编写程序，可以实现对问题的计算求解，这个过程是一种计算方法：流程表达法。

流程表达法简单直接，适用于所有计算问题的求解。当问题不能够表达为明确流程时，该问题也是计算不可解的。当问题能够表达为明确流程时，也可能受限于计算机速度或程序复杂度而在计算上不可解。

流程表达法以步骤描述、伪代码、流程图等人们能理解的流程表达形式为输入，不限定具体输入方式，但要求输入所表达的步骤具体明确。进而，逐个步骤编写代码，对于不能明确实现过程的步骤，再进一步设计算法或进行流程拆分。最终，输出与流程相一致的可执行程序代码。

对于初学编程的读者，首先建立 IPO 程序编写方法的认知，区分程序输入、处理和输出过程。进一步，对于处理阶段，将问题表达为一系列具体的求解步骤，通过流程图或伪代码，尽可能细致地描述求解步骤，表达计算流程。最后，根据流程逐步编写代码，完成程序设计。

流程表达法是最简单的计算方法，不再赘述。本书后续将陆续介绍一批计算方法，包括：随机采样法（4.6.3 节）、递归分解法（5.4.3 节）、框架填充法（8.5.3 节）、对象构造法（9.1.3 节），帮助读者建立基本的计算求解思路。

思考与练习 1-5



思考与练习

1.13 一个程序的控制结构可能有哪几种？

1.14 简述通过计算思维解决问题的基本过程。

1.15 下列描述中，属于计算思维应用的是（　　）。

A. 通过多次实验与统计，总结事件发生的规律

B. 通过复杂的推导，验证了数学公式的正确性

C. 高考中，算出了一道很难的数学题

D. 对一类问题进行数学建模，并通过程序解决问题

1.6　Python 程序认知

要点： 规范、严谨是编写程序所需要的基本态度。

1.6.1　运行 Python 程序

本节给出 5 个 5 行代码左右的 Python 小程序，供读者采用 IDLE 交互式和批量式运行方式练习。

请读者暂时忽略这些程序的具体语法含义，这是后续章节要介绍的内容，当然，尝试理解语法也十分有益。请在 IDLE 交互环境或编辑器中编写并运行这些程序，确保它们可以输出正确结果。

注意：在编辑器中输入代码时，# 及后面的文字可以不用输入，它们是注释，仅用来帮助读者理解程序，不影响程序执行。

【微实例 1A】 圆面积的计算。

根据圆半径计算圆的面积。启动 IDLE 工具，逐行输入以下代码，其中，>>> 为交互提示符，请在这个提示符的后面输入代码。交互式运行过程如下：

```
>>> radius = 25                          # 圆的半径是 25
>>> area = 3.1415 * radius * radius      # 输入计算圆面积的公式
>>> print(area)                          # 输出结果
1963.4375000000002
>>> print("{:.2f}".format(area))          # 输出结果，仅保留两位小数
1963.44
```

在 IDLE 菜单中选择“File → New File”选项，或使用快捷键 Ctrl+N 打开一个新的编辑窗口，在其中进行代码编写，输入微实例 1A 的文件式运行内容，如下。其中，首行实例文件名对应本书电子资源文件名，读者可以将代码保存为任意名称，代码中左侧序号为行号，以辅助阅读，不是程序代码的组成部分。

代码 1A　　CalCircleArea.py

```
1 radius = 25
2 area = 3.1415 * radius * radius
3 print(area)
4 print("{:.2f}".format(area))
```

源代码 1-2：圆面积的计算



在菜单中选择“Run → Run Module”选项，或按快捷键 F5，可以执行该文件。IDLE 启动一个交互环境显示输出结果，如下：

```
1963.4375000000002
1963.44
```

【微实例 1B】 简单的人名对话。

对用户输入的人名给出一些不同的回应。交互式运行过程如下：

```
>>> name = input("输入姓名：")
输入姓名：郭靖
>>> print("{}同学，学好 Python，前途无量！".format(name))
郭靖同学，学好 Python，前途无量！
>>> print("{}大侠，学好 Python，大展拳脚！".format(name[0]))
郭大侠，学好 Python，大展拳脚！
>>> print("{}哥哥，学好 Python，人见人爱！".format(name[1:]))
靖哥哥，学好 Python，人见人爱！
```

在 IDLE 菜单中选择“File → New File”选项，或使用快捷键 Ctrl+N 打开一个新的编辑窗口，在其中进行代码编写，输入微实例 1B 的文件式运行内容如下：

源代码 1-3：简单的人名对话



代码 1B EchoName.py

```
1 name = input("输入姓名：")
2 print("{}同学，学好 Python，前途无量！".format(name))
3 print("{}大侠，学好 Python，大展拳脚！".format(name[0]))
4 print("{}哥哥，学好 Python，人见人爱！".format(name[1:]))
```

在菜单中选择“Run → Run Module”选项，或按快捷键 F5，可以执行该文件。IDLE 启动一个交互环境显示输出结果如下：

```
输入姓名：郭靖
郭靖同学，学好 Python，前途无量！
郭大侠，学好 Python，大展拳脚！
靖哥哥，学好 Python，人见人爱！
```

【微实例 1C】 斐波那契数列的计算。

根据斐波那契数列的定义，输出不大于 1 000 的序列元素。交互式运行过程如下：

```
>>> a, b = 1, 1
>>> while a < 1000:                    # 输出不大于 1000 的序列
...     print(a, end=',')              # print 用于输出一个数字，但不换行
...     a, b = b, a + b
...

1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,
```

上述代码中，在交互提示符（>>>）下方的 3 个连续点（…）是二级输入提示符，表示所在行延续上一行内容。这个二级输入提示符在 IDLE 环境中可能出现或不出现，不同 IDLE 版本略有不同。

拓展： 斐波那契数列

斐波那契数列（Fibonacci sequence），又称为黄金分割数列，由意大利数学家 Leonardo Fibonacci 于 1202 年提出，以其名字命名。该数列 F(n) 定义为：F(0)=1，F(1)=1，F(n)=F(n−2)+F(n−1)，其中，$n \geqslant 2$。简单说，斐波那契数列中每个数是前两个数之和。斐波那契数列中邻近两个数的比值接近黄金分割数，即 F(n)/F(n−1) 接近 1.618，这个比例的极限值就是黄金分割数。

斐波那契数列有很多特性，在搜索算法、组合数学、现代物理、化学等领域均有应用。Python 语言在表达和计算传统数学概念上十分简洁，读者可以寻找其他有趣的数学问题，并用 Python 语言计算它们。

在 IDLE 菜单中选择“File → New File”选项，或使用快捷键 Ctrl+N 打开一个新的编辑窗口，在其中进行代码编写，输入微实例 1C 的文件式运行内容如下：

代码 1C　　CalFibonacci.py

源代码 1-4：斐波那契数列的计算



```
a, b = 1, 1
while a < 1000:
    print(a, end=',')
    a, b = b, a + b
```

在菜单中选择“Run → Run Module”选项，或按快捷键 F5，可以执行该文件。IDLE 启动一个交互环境显示输出结果如下：

```
1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,
```

【微实例 1D】 同切圆的绘制。

绘制 4 个不同半径的同切圆。交互式运行过程如下：

```
>>> import turtle                    # 引用 turtle 库
>>> turtle.pensize(2)                # 设置画笔宽度为 2 像素
>>> turtle.circle(10)                # 绘制半径为 10 像素的圆
>>> turtle.circle(40)                # 绘制半径为 40 像素的圆
>>> turtle.circle(80)                # 绘制半径为 80 像素的圆
>>> turtle.circle(160)               # 绘制半径为 160 像素的圆
```

随着每条语句的执行，IDLE 会启动一个窗体，显示如图 1.18 所示的图形。

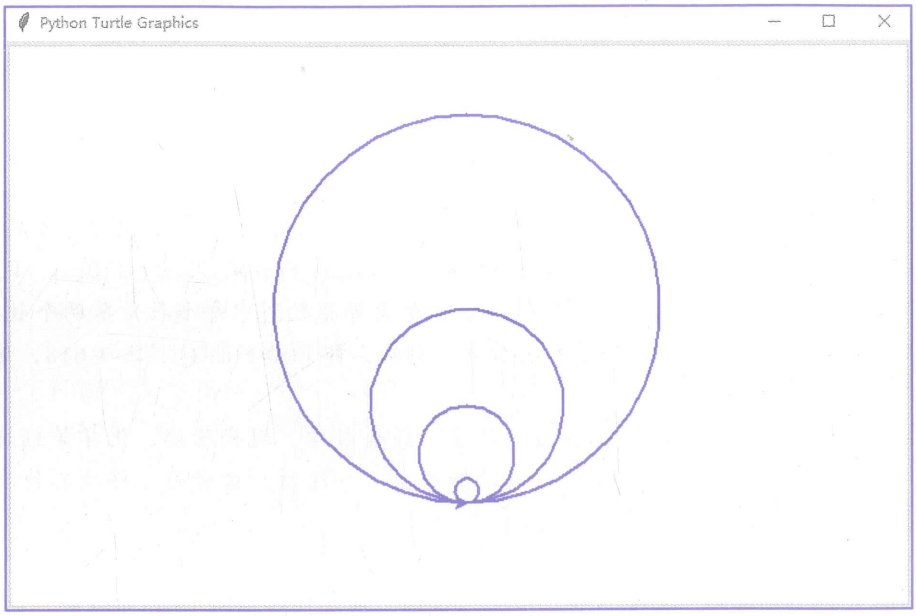


图 1.18　实例 1D 运行后输出的一组同切圆

在 IDLE 菜单中选择“File → New File”选项，或使用快捷键 Ctrl+N 打开一个新的编辑窗口，在其中进行代码编写，输入微实例 1D 的文件式运行内容如下：

源代码 1-5：同切圆的绘制



代码 1D　　DrawTangentCircles.py

```
1  import turtle
2  turtle.pensize(2)
3  turtle.circle(10)
4  turtle.circle(40)
5  turtle.circle(80)
6  turtle.circle(160)
```

在菜单中选择“Run → Run Module”选项，或按快捷键 F5，可以执行该文件。IDLE 将启动一个窗体，显示如图 1.8 所示的图形。

【微实例 1E】日期和时间的输出。

输出当前计算机的系统日期和时间。交互式运行过程如下：

```
>>> from datetime import datetime          # 引用 datetime 库
>>> now = datetime.now()                   # 获得当前日期和时间信息
>>> print(now)                             # 输出当前时间
2023-03-05 20:46:03.059649
>>> print(now.strftime("%x"))              # 输出当前时间的日期部分
03/05/23
>>> print(now.strftime("%X"))              # 输出当前时间的时分秒部分
20:46:03
```

在 IDLE 菜单中选择“File → New File”选项，或使用快捷键 Ctrl+N 打开一个新的编辑窗口，在其中进行代码编写，输入微实例 1E 的文件式运行内容如下：

代码 1E　　PrintLocalTime.py

```
from datetime import datetime
now = datetime.now()
print(now)
print(now.strftime("%x"))
print(now.strftime("%X"))
```

源代码 1-6：日期和时间的输出



在菜单中选择“Run → Run Module”选项，或按快捷键 F5，可以执行该文件。IDLE 启动一个交互环境显示输出结果如下：

```
2023-03-05 20:46:03.059649
03/05/23
20:46:03
```

这个程序的每次运行都会显示不一样的时间结果，正所谓：一日难再晨，及时当勉励，岁月不待人。

1.6.2　常见的程序问题

规范、严谨是编写程序所需要的基本态度。

程序执行严格遵循代码逻辑，不能容错，即使微小符号编写错误，也将导致程序无法执行。初学 Python 语言经常会遇到程序无法运行等各类异常问题。这里简要列出 5 类常见问题及解决方法，帮助读者度过初识 Python 语言的起步阶段。

问题 1：Python 安装失败。

在安装 Python 解释器时，如果出现如图 1.19 所示的情况，窗口提示“Setup failed”，表明安装失败。

安装失败有多种可能性，主要原因是当前操作系统与 Python 版本不兼容，一般情况是在较旧版本 Windows 系统中安装了较新的 Python 解释器，例如，在未安装 Service Pack 补丁的 Windows 7 系统上安装 Python 3.5 及以上版本解释器将会失败。此时，建议安装较低版本 Python 解释器，如在 Windows 7 系统上安装 Python 3.4.2 或更低版本，但不要低于 Python 3.3.1 版本。在安装 Python 解释器时，建议首先选择 64 位版本安装，若无法安装，再选择 32 位版本进行尝试。64 位版本和 32 位版本在程序执行性能方面略有不同，但在程序语法和应用方面无差别。

问题 2：Python 代码完备错误。

在 IDLE 交互式或文件式模式下，可能出现“unterminated string literal”错误信息提示，这是 Python 代码完备性错误，如图 1.20 所示。

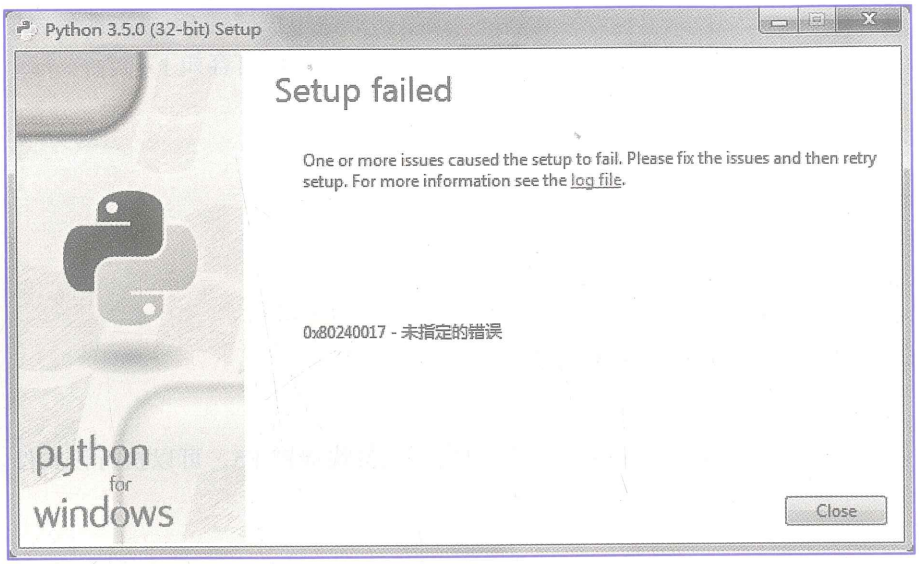


图 1.19 Python 安装失败的情况

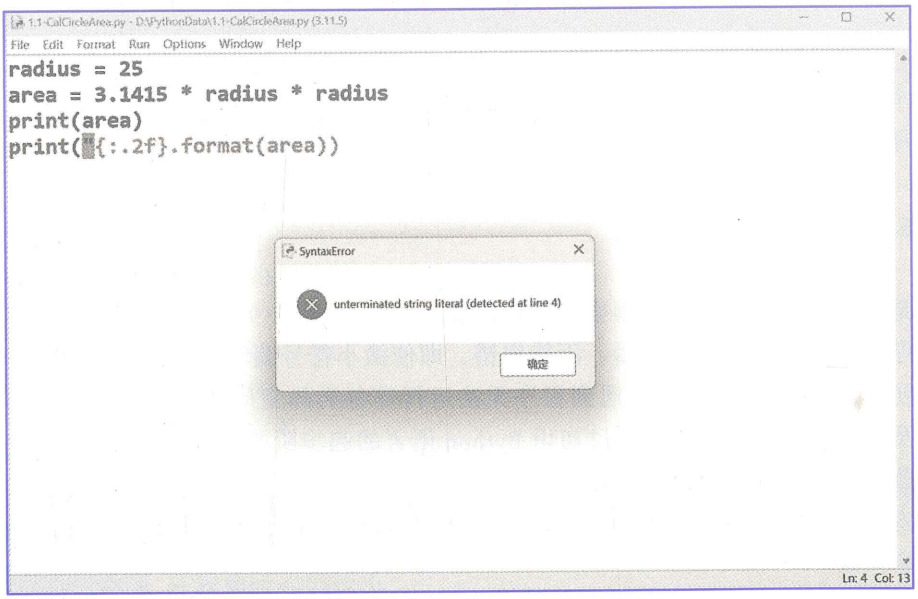


图 1.20 Python 代码完备错误

这类错误原因在于代码编写不完备，导致解释器不能理解程序。图 1.20 中，代码第 4 行只有一个双引号（"），而 Python 语法要求表达字符串的双引号成对出现，因为输入程序少了一个双引号，因此产生错误。这类错误具体成因与所用编程语法有关，要结合具体代码分析，通常多次检查语法输入是否正确可解决这类问题。

问题 3：Python 程序缩进错误。

在 IDLE 交互式或文件式模式下，可能出现“unexpected indent”错误，造成 Python 程序缩进错误，如图 1.21 所示。

这类错误正如英文提示所描述，出现了未知缩进。在图 1.21 中，第 2 行语句前多了一

个空格，产生了错误的缩进，因此程序运行出错。一般来说，只要提示中包含“indent”，都是由于缩进不正确造成的，需要检查输入程序的缩进是否一致，需要特别注意，在 Python 语言中，缩进是语法的一部分。

问题 4：Python 语法错误。

在 IDLE 文件式下，执行 Python 程序出现“invalid syntax”错误，造成 Python 语法错误，如图 1.22 所示。

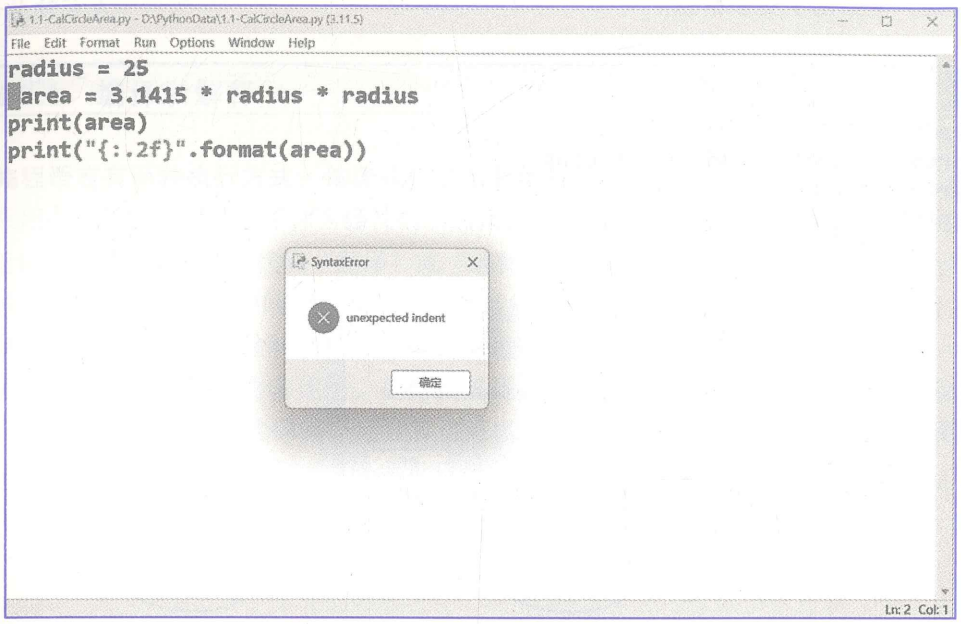


图 1.21　Python 程序缩进错误

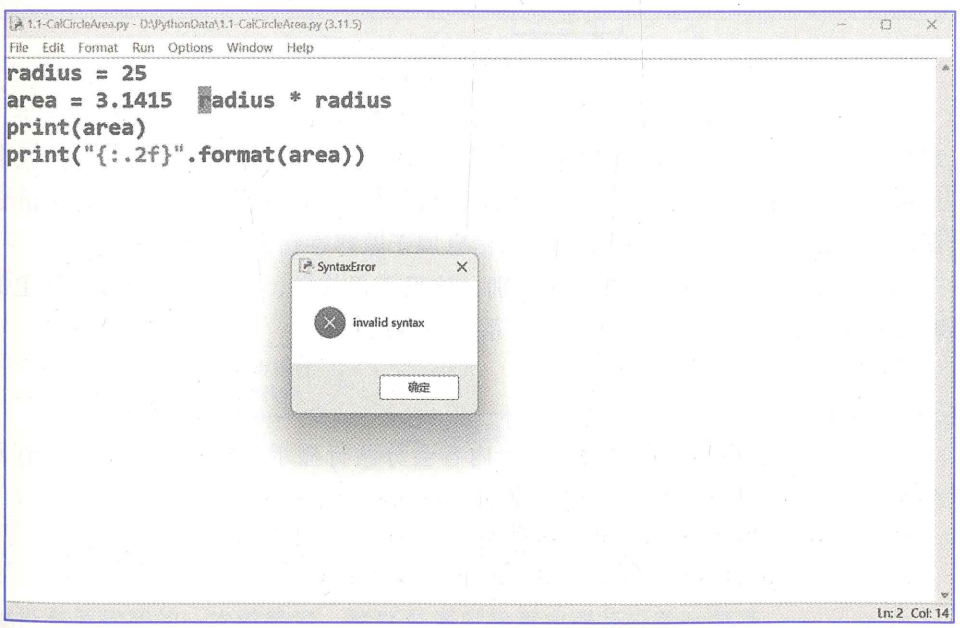


图 1.22　Python 语法错误

这类错误出现与语法格式错误有关。在图 1.22 中，3.141 5 与 radius 变量之间缺少操作符，导致语法错误。一般来说，只要提示中包含单词“syntax”，都是语法格式错误，需要检查程序中语法使用是否正确。这类错误有时候与问题 2（Python 代码完备错误）所反映的问题相似，具体需要结合错误提示信息来调试。

问题 5：Python 版本兼容错误。

这种错误的一种情况如图 1.23 所示，虽然程序提示了“Syntax Error”，即语法错误，但给出了更多信息。

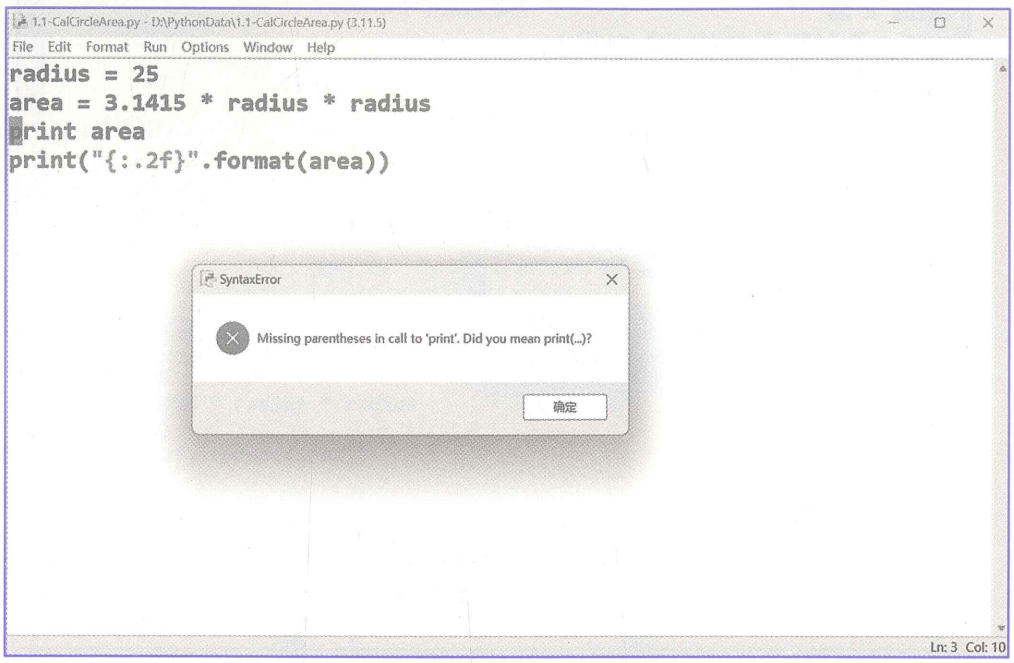


图 1.23 Python 版本兼容错误

Python 语言有 3.x 和 2.x 两个版本系列，它们不兼容，虽然 2.x 不再演进，但互联网上仍有大量的 2.x 版本代码。这些代码的一个重要标志是 print 的用法。在 Python 2.x 系列中，print 后面的输出变量没有括号，而 Python 3.x 系列将 print 定义为函数，即 print()。图 1.23 所示的提示信息属于兼容性错误。

Python 2.x 与 3.x 系列存在较多差别，这里不再枚举，读者在学习互联网上的代码时务必看清版本，请使用 Python 3.x 系列版本代码。

思考与练习

思考与练习 1-6

1.16 两个连续 print() 函数的输出内容会分两行显示，即调用一次 print() 函数后会换行。如何让两个连续 print() 函数输出的内容在一行内？

1.17 import 保留字用来引入功能库，绘制图形使用什么 Python 功能库？

1.18 获得系统日期和时间使用什么 Python 功能库？

1.7　计算机编程

微视频 1-8：计算机编程



要点： 编程语言有两种执行方式：编译执行和解释执行，Python 语言主要采用解释执行方式

1.7.1　编译和解释

编程语言有两种执行方式：编译执行和解释执行。

采用高级语言编写的程序称为**源代码**。然而，计算机不能直接执行源代码，需要通过一个中间程序将高级语言的源代码“翻译”或“解释”成机器语言程序，进而在计算机上执行。根据转换方式不同，这个中间程序叫作**编译器**或**解释器**，表现为两种不同的程序执行方式。

编译是将源代码一次性转换成目标代码的过程。

通常，源代码是高级语言代码，目标代码是机器语言代码，执行编译的计算机程序称为编译器（compiler），源代码编译后计算机可以立即或稍后执行相应的目标代码。图 1.24 展示了程序的编译过程，其中，虚线表示目标代码被计算机执行。

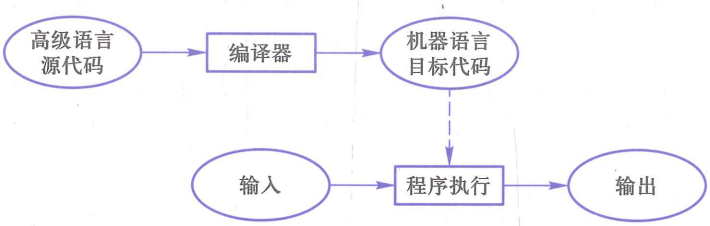


图 1.24　程序的编译和执行过程

解释是将源代码逐条转换成目标代码同时逐条运行的过程。

执行解释的计算机程序称为解释器（interpreter），经过解释产生的目标代码可以逐条运行。图 1.25 展示了程序的解释过程。其中，高级语言源代码与数据一同输入给解释器，然后输出运行结果。

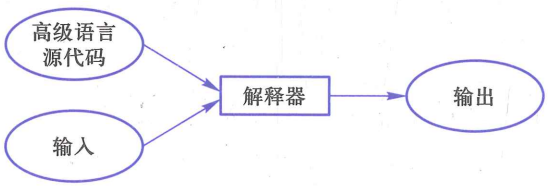


图 1.25　程序的解释和执行过程

编译是源代码的一次性翻译，解释是源代码每次逐行的实时翻译。

解释和编译的区别在于编译是一次性的翻译，一旦程序被编译，就不再需要编译器或

者源代码。解释则在每次程序运行时都需要解释器和源代码。这两者的区别类似于外语资料的翻译和现场的同声传译。

编译过程只进行一次，所以，编译过程的速度并不是关键，目标代码的运行速度是关键。因此，编译器一般都集成尽可能多的优化技术，使生成的目标代码具备更高的执行效率。然而，解释器却不能集成太多优化技术，因为代码优化技术会消耗运行时间，使整个程序的执行速度受到影响。

采用编译方式有如下好处：

（1）对于相同源代码，编译所产生的目标代码执行速度更快。

（2）目标代码不需要编译器就可以运行，在同类型操作系统上使用灵活。

采用解释方式有如下好处：

（1）解释执行需要保留源代码，程序维护十分方便。

（2）只要存在解释器，源代码可以在任何操作系统上运行，可移植性好。

简单地说，解释执行、逐条运行用户编写的代码，没有纵览全部代码的性能优化过程，因此执行性能略低，但可以支持跨硬件或操作系统平台执行，保留源代码对升级维护十分有利，适用于非性能关键的程序运行场景。

编程语言按照执行方式的不同可分成两类：静态语言和脚本语言。

采用编译执行的编程语言是静态语言，如 C 语言、Java 语言；采用解释执行的编程语言是脚本语言，如 JavaScript 语言、PHP 语言。Python 语言是一种通用编程语言，是脚本语言，虽采用解释执行方式，但它的解释器也保留了编译器的部分功能，随着程序运行，解释器也会生成一个完整的目标代码。这种将解释器和编译器结合的新解释器是现代脚本语言为了提升计算性能的一种有益演进。无论哪种执行方式，用户对高级语言的使用方法可以是一致的，例如，通过鼠标双击执行一个程序。

拓展：通用编程语言

通用编程语言是一个相对概念，指能够用于编写多种用途程序的编程语言。例如，Python 语言是一个通用编程语言，因为该语言没有专门用于特定领域的语法元素。HTML 语言则是一个专用编程语言，它利用超链接将文本、图像、音 / 视频等资源组织起来形成 Web 页面，语法中提供了超链接的组织。尽管有些编程语言不包含特定应用元素，但由于应用领域比较狭窄，也被认为是专用编程语言。

常用编程语言中，C、C++、C#、Go、Java、Python 是通用编程语言，HTML（Web 页面超链接语言）、JavaScript（Web 浏览器端动态脚本语言）、MATLAB（基于矩阵运算的科学计算语言）、PHP（Web 服务器端动态脚本语言）、SQL（数据库操作语言）、Verilog（硬件描述语言）是专用编程语言。

1.7.2 为什么学编程

——为什么要学习计算机编程？

——因为“编程”是件很有趣的事儿，能够启迪思维，认识世界。

2022 年 11 月，美国 OpenAI 公司发布在线自然语言大模型：ChatGPT，它能够通过理解和学习人类的语言进行对话，真正像人类一样聊天交流，并能够完成各类型文本撰写任务，甚至可以对简单计算问题进行自动编程求解。或许，读者会问，既然 ChatGPT 这类工具已经能够提供简单的自动编程了，为何还要学习编程呢？

人生是一场修炼，学习编程并不只为了掌握工作技能这个目标，还要通过它认识更广阔的世界，因为学习编程是一种独特的经历。

编程能够训练思维、增进认识、带来乐趣、提高效率、创造机会，而且不难学。

（1）编程能够训练思维。

编程体现了一种抽象交互关系、形式化执行的计算思维模式，这是区别于以数学为代表的逻辑思维和以物理为代表的实证思维的第三种思维模式。编程也是一个求解问题的过程，首先需要分析问题，抽象内容之间的交互关系，设计利用计算机求解问题的确定性方法，进而通过编写和调试代码解决问题，这是从抽象问题到解决问题的完整过程。计算思维的训练过程能够促进人类思考，增强观察力和加深对交互关系的理解。

（2）编程能够增进认识。

编写程序不单纯是求解计算题，它要求作者不仅要思考解决问题的方法，更要思考如何让程序提供更好的用户体验、更高的执行效率和更有趣的展示效果。不同群体、不同时代、不同文化对程序使用有着不同理解，编程需要对时代大环境和使用群体小环境有更多认识，从细微处给出更好的程序体验，这些思考和实践将帮助程序作者加深对用户行为以及社会和文化的认识。

（3）编程能够带来乐趣。

利用一台计算机，编程能够提供展示自身思想和能力的舞台，将所思所想变为现实。编程的开始有各种动机，或者去展示自己的青春风采，或者解决不高效的任务流程，或者向爱慕的对象表达情愫，所有这些想法都可以通过编写程序变成现实，并通过互联网零成本分享获得更大的影响力。这些努力会让世界增加新的颜色，让自己变得更酷，提升心理满足感和安全感。

（4）编程能够提高效率。

计算机已经成为当今社会的普通工具，掌握一定编程技术有助于更好地利用计算机解决所面对的各种问题。例如，对于个人照片，可以通过程序读取照片元属性并将它们自动整理归类；对于工作数据，可以通过程序按照特定算法进行批处理并绘制统计图表；对于朋友圈的好文，可以通过程序实时关注随时点赞。掌握一些编程技术能够提高工作、生活和学习效率。

（5）编程创造就业机会。

程序员是信息时代最重要的工作岗位之一，在建设数字中国、推动数字经济发展的国家战略下，程序员岗位缺口在百万级规模，就业前景广阔。随着信息化进程深入，各行各业也都有大量数字化演进的实际需求，会编程已然是高质量就业的重要条件。会编程并不需要掌握多种编程语言，精通一种或几种就够了，但无论精通哪种编程语言，Python 语言则是必须掌握的。请认真学习 Python，好工作就在前面！

很多读者都有一个误区，认为编程很难学。事实上，“学习编程并不是一件很难的事情”，因为编写程序有一定的框架和模式，只要理解这些模式，稍加练习就会有很好的学

习效果。学习一门编程语言，首先要掌握该语言的语法，对它们既要系统掌握，又要灵活运用。其次要学会结合计算问题设计程序结构，从程序块、功能块角度理解并设计整个程序框架。最后要掌握解决问题的设计和实践能力，即从理解计算问题开始，设计问题的解决方法，并通过编程语言来实现。

学习计算机编程的重点在于练习。不仅要多看代码，照着编写，调试运行，还要在参考代码思路基础上独立编写，学会举一反三。为了帮助读者掌握程序设计能力，本书将大量编程概念和语法通过有趣的实例组织起来，展示 Python 语言的魅力和力量。希望这样的设计能够为读者的 Python 学习过程带来快乐和价值。

思考与练习 1-7



思考与练习

1.19 请阐述编译和解释两种执行方式的区别和各自优缺点。

1.20 Python 是支撑人工智能应用的基础性编程语言，各行各业都逐步设定“智慧”目标，如智慧社会、智慧城市，请列出不少于 3 个冠以“智慧”的新名词。

1.21 结合实际情况，请列出不少于 3 个学习编程的理由。

本章小结

本章具体讲解了计算机的基本概念、计算机技术发展历程及 4 个阶段、Python 语言发展及特点、程序设计语言分类、Python 开发环境配置、IPO 程序编写方法、问题的求解步骤、问题的计算性分析、算法基本概念、计算思维、编译和解释等内容，通过多个小程序帮助读者感性认识 Python 程序，了解学编程的价值，建立“程序设计”认识，期待读者“看见”更大的世界。

程序练习题：第 1 章



程序练习题

请读者输入并正确运行下列程序，对 Python 语言有个初步认识，结合 1.3 节介绍的内容，熟练掌握 Python 开发环境，此刻可暂不用理解代码中的语法含义。

1.1 字符串拼接。接收用户输入的两个字符串，将它们组合后输出。

源代码 1-7：字符串拼接



```
str1 = input("请输入一个人的名字:")
str2 = input("请输入一个国家名字:")
print("世界这么大,{}想去{}看看.".format(str1,str2))
```

1.2 整数序列求和。用户输入任意一个正整数 N，计算从 1 到 N（包含 1 和 N）相加之后的结果。

```
n = input("请输入整数 N:")
sum = 0
for i in range(int(n)):
    sum += i + 1
print("1 到 N 之和 :", sum)
```

源代码 1-8：整数序列求和



1.3　九九乘法表输出。工整打印输出常用的九九乘法表。

```
for i in range(1,10):
    for j in range(1,i+1):
        print("{}*{}={:2} ".format(j,i,i*j), end='')
    print('')
```

源代码 1-9：九九乘法表输出



1.4　阶乘序列求和。计算 1+2!+3!+…+10! 的结果。

```
sum, tmp = 0, 1
for i in range(1,11):
    tmp *= i
    sum += tmp
print("运算结果是 : {}".format(sum))
```

源代码 1-10：阶乘序列求和



1.5　健康食谱输出。列出 5 种不同食材，输出它们两两组合炒制可能组成的所有菜品名称。

```
diet = ['西红柿','花椰菜','黄瓜','牛柳', '虾仁']
for x in range(0, 5):
    for y in range(0, 5):
        if not(x == y):
            print("{} 炒 {}".format(diet[x], diet[y]))
```

源代码 1-11：健康食谱输出



1.6　幸运数字计算。用户输入姓名，输出该名字对应的幸运数字，不超过 3 位数字。注意：所谓“幸运数字”，无非是一种数字计算游戏。

```
import random
name = input("请输入名字 :")
lucky = 1
random.seed(7)
for c in name:
    lucky += ord(c)*random.randint(1,77)
print("幸运数字是 :{}".format(lucky%777))
```

源代码 1-12：幸运数字计算



1.7 五角星绘制。绘制一个红色的五角星图形，如图 1.26 所示。

源代码 1-13：五角星绘制



```
from turtle import *
fillcolor("red")
begin_fill()
while True:
    forward(200)
    right(144)
    if abs(pos()) < 1:
        break
end_fill()
```

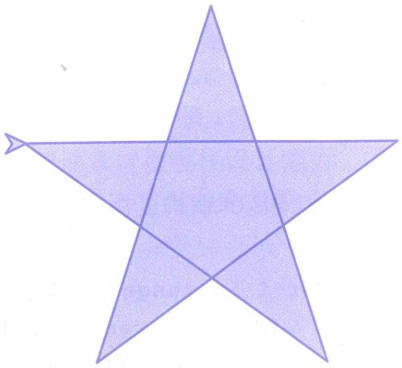


图 1.26 五角星的绘制结果

1.8 太阳花绘制。绘制一个太阳花的图形，如图 1.27 所示。

源代码 1-14：太阳花绘制



```
from turtle import *
color('red', 'yellow')
begin_fill()
while True:
    forward(200)
    left(170)
    if abs(pos()) < 1:
        break
end_fill()
done()
```

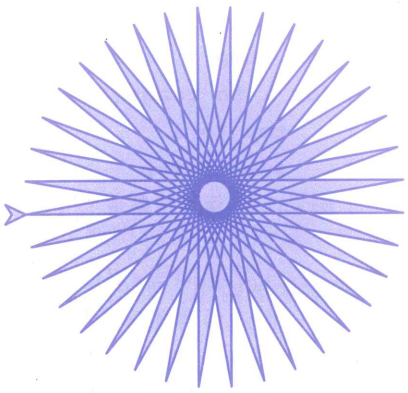


图 1.27 太阳花的绘制结果